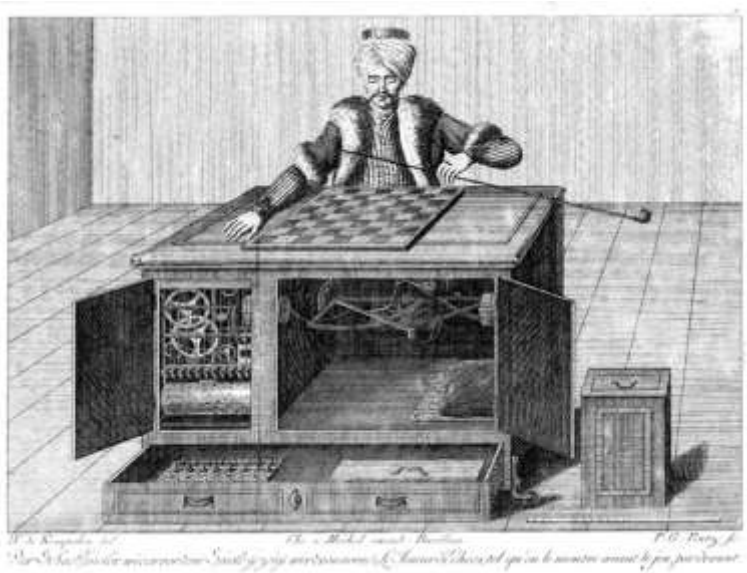
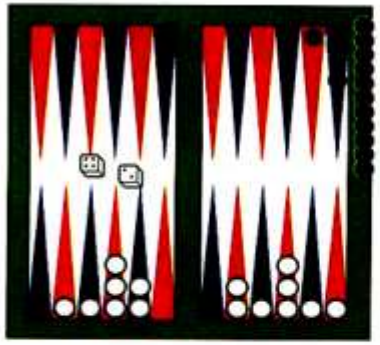
Artificial Intelligence (CS 1360)
Games: Adversarial Search



This slide from: ai.berkeley.edu with slight modifications.

Games: Outline of Unit

- Game playing
- Game trees
  - MinMax procedure
  - Alpha-beta pruning
- Examples



BACKGAMMON

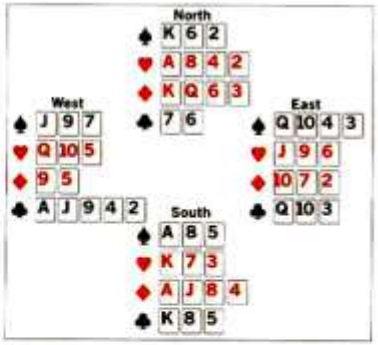
- 2 players
- 15 pieces each
- Goal: Move all pieces off the board
- Rules:

  Dice roll determines number of moves

  Players move in opposite directions

  Piece cannot land on a point occupied by 2 or more of opponent's pieces

  Single piece can be "hit" if landed on by opponent; hit piece must start anew
- Program: TD-Gammon*
- Web site: www.research.ibm.com/massdist/tdl.html
- Advantage: Too close to call



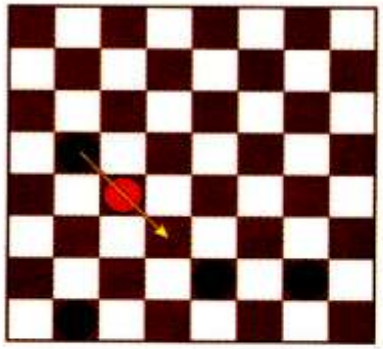
BRIDGE

- 4 players in 2 teams
- 13 cards dealt to each player
- Goal: Make 2 "game contracts," or a "rubber"
- Rules:

  The bid: Each player predicts how many times his or her card will be the highest (a trick)

  The play: Put down 1 card at a time and compare it with others; this occurs 13 times

  The scoring: Points scored if bid is made or exceeded; otherwise points go to the opposing team
- Program: GIB*
- Web site: www.gibware.com
- Advantage: Human



CHECKERS

- 2 players
- 12 pieces each
- Goal: Avoid being the player who can no longer move (usually when a player has no pieces left)
- Rules:

  Move forward on dark diagonal, 1 square at a time

  Opponent's piece captured when jumped to empty square diagonally behind opponent's piece

  Creation of a "king," a piece that can move backward and forward, occurs when piece is moved to opponent's last row
- Program: Chinook
- Web site: www.cs.ualberta.ca/~chinook
- Advantage: Machine

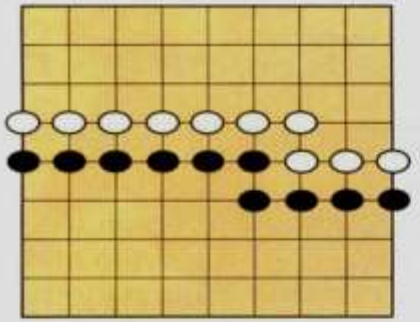


CHESS

- 2 players
- 16 pieces each (1 king, 1 queen, 2 rooks, 2 bishops, 2 knights, 8 pawns)
- Goal: Capture opponent's king (checkmate)
- Rules:

  Pieces are captured when landed on by opponent's piece

  Type of piece dictates movement options
- Program: Deep Blue
- Web site: www.chess.ibm.com/meet/html/d.3.html
- Advantage: Too close to call

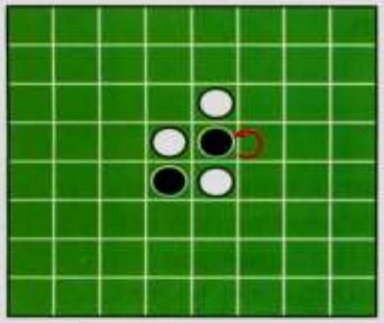


GO

- 2 players
- Black-and-white stones
- Grid size of board can vary: typical game is on 19-by-19 grid points
- Goal: Conquer a larger part of the board (conquered part encompasses stones placed on board plus stones that could be added safely—that is, within the player's walls)
- Rules:

  Both sides alternate in placing stones on the board

  Stones surrounded by an opponent's stones are captured and removed from the board
- Program: Handtalk*
- Web site: www.webwind.com/go
- Advantage: Human, by a huge margin

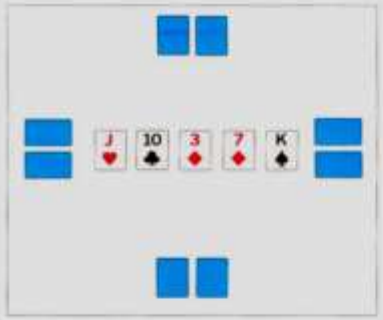


OTHELLO

- 2 players
- Black-and-white disks
- Goal: Have most disks on the board at the end of the game
- Rules:

  Players alternate placing disks on unoccupied board spaces

  If opponent's disks are trapped between other player's disks, opponent's disks are flipped to the other player's color
- Program: Logistello
- Web site: www.neci.nj.nec.com/homepages/mic/log.html
- Advantage: Machine



POKER (Texas Hold 'Em)

- 3 to 20 players
- 2 cards dealt to each player; 5 cards placed in center of table
- Goal: Obtain the best hand and win the "pot"
- Rules:

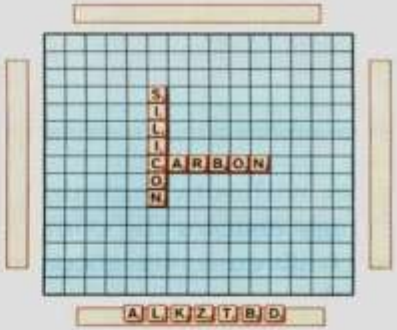
  5 center (community) cards start facedown

  First round of betting ensues; 3 community cards are turned over

  Subsequent rounds of betting ensue; 4th and 5th community cards turned over

  Players select best 5 from the community cards and their hands to obtain identical kinds of cards (pairs, 3- and 4-of-a-kind), flushes (all same suit), straights (sequential) or their combinations

  Final round of betting ensues
- Program: LOKI
- Web site: www.cs.ualberta.ca/~games/poker
- Advantage: Human, by a huge margin



SCRABBLE

- 2 to 4 players
- 100 tiled letters
- Goal: Accumulate most points by creating high-scoring words
- Rules:

  Each player draws 7 letters

  Each letter has a value

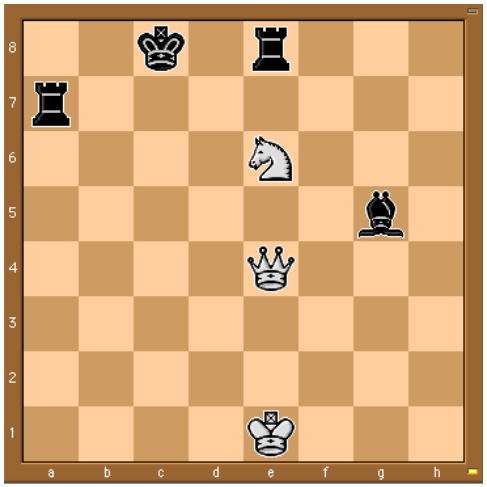
  Squares on the board have values

  Words created must join an array
- Program: Maven* (used in Scrabble CD-ROM)
- Web site: www.hasbroscrabble.com/cd/cd.html
- Advantage: Machine, by a slight margin

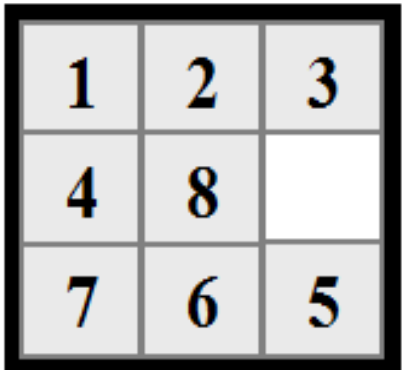
*Indicates commercial software that runs on personal computers

Motivation

- Games are a form of *multi-agent environment*
  - What do other agents do and how do they affect our success?
  - Cooperative vs. competitive multi-agent environments.
  - Competitive multi-agent environments give rise to adversarial search, often known as games



Adversarial



Not adversarial

Motivation

- Why study games?
  - Games are fun!
  - Historical role in AI
  - Studying games teaches us how to deal with other agents trying to fail our plans
  - *Huge* state spaces – Games are *hard*!
  - Nice, clean environment with clear criteria for success

What is the Type of Game?

- How many players?
- Is it zero-sum (one player's loss is another's gain)?
- Is it fully observable or partially observable?
- Is it deterministic or stochastic?

The Simple Case

- **Examples:** Chess, Checkers, Go, Mancala, Tic-Tac-Toe, Othello, Nim, …
- **Two players** alternate moves (turn-taking)
- **Zero-sum**
- **Perfect Information:** each player knows the entire game state (fully observable).
- **Deterministic:** no element of chance
- Clear set of **legal moves.**
- **Well-defined outcomes** (e.g. win, lose, draw)

More Complicated Games

- Most card games (e.g. Hearts, Bridge, etc.) and Scrabble
  - **non-deterministic**
  - **lacking** in perfect information.
- **Cooperative** games
- **Real-time strategy** games

Zero Sum Games





- Zero-Sum Games
  - Agents have opposite utilities
  - Lets us think of a single value that one maximizes and the other minimizes.
  - Adversarial, pure competition.
  - if one player wins a game, the other player necessarily loses.
- General Games
  - Agents have independent utilities
  - Cooperation, indifference, competition, and more are all possible

How to Play a Game by Searching

- A way to play such a game is to:
  1. Consider **all the legal** moves you can make
  2. Compute the **new position** resulting from each move
  3. **Evaluate** each resulting position and determine which is best
  4. Make that **move**
  5. **Wait** for your opponent to move and repeat

- **Key problems** are:
  - Representing the "board"
  - Generating all legal next boards
  - Evaluating a position

Game setup

- Two players: ***A*** and ***B*** (MAX and MIN)
- ***A*** (MAX) moves first, and they take turns until the game is over.
- Winner gets award, loser gets penalty.
- **Games as search**:
  - **Initial state:** specifies how the game is set up at the start. e.g. board configuration of chess
  - **Successor function:** list of (*move, state*) pairs specifying legal moves.
  - **Terminal test**: A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
  - **Utility function**: Gives numerical value of terminal states.
    - in tic-tac-toe win (+1), lose (-1) and draw (0)
    - In chess, the outcome is a win, loss, or draw, with values +1, 0, or 1/2 .

- **A** uses search tree to determine **next move.**

Game Trees

- The initial state, action function, and successor function define the game tree.

- **Game tree** represents the problem **space** for a game by a **tree:**
  - **Nodes** represent game states, e.g., 'board positions';
  - **Edges** represent legal moves.

- **Root node** is the position in which a decision must be made.

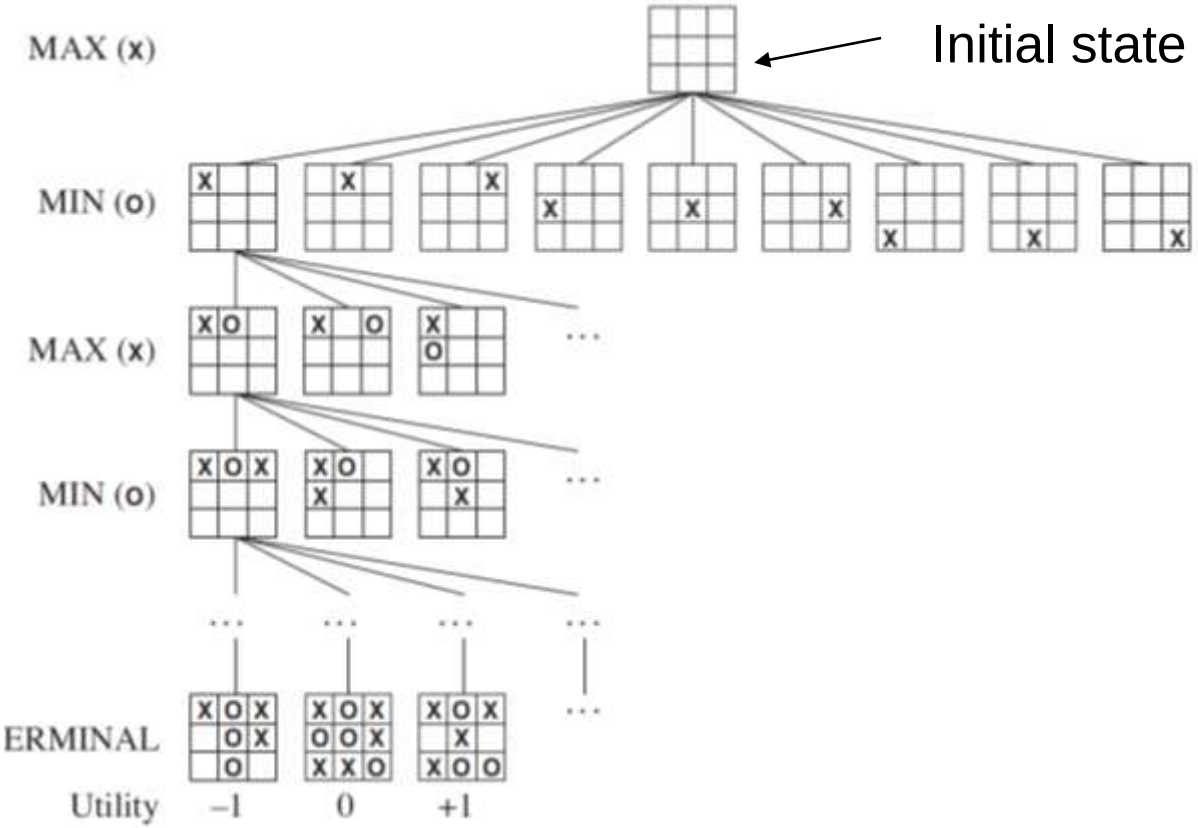
- **Evaluation function** *f* assigns real-number scores to `board positions.'

Game setup

- Example: A (partial) game tree for the game of tic-tac-toe.

- MAX moves first, placing an X in an empty square.

- Alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states.



Game Trees

- **Terminal nodes** represent ways the game could end, labeled with the desirability of that ending (e.g. win/lose/draw or a numerical score).
  - terminal states are assigned utilities according to the rules of the game.
  - The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names).

- The term **search tree** is used for a tree that is superimposed on the full game tree, and examines enough nodes to allow a player to determine what move to make.

MAX & MIN Nodes

- When I move, I attempt to **MAXimize** my performance.
- When my opponent moves, he attempts to **MINimize** my performance.

TO REPRESENT THIS:

- If we move **first**, label the root **MAX**;
- if our **opponent** does, label it **MIN**.

- **Alternate labels** for each successive tree level.
  - if the root (level 0) is our turn (MAX)
  - all EVEN levels will represent turns for us (MAX)
  - and all ODD ones turns for our opponent (MIN)

Optimal Decisions in Games

- In a normal search
  - the optimal solution would be a sequence of actions leading to a goal state—a terminal state that is a win.

- In adversarial search
  - MIN has something to say about it.
  - MAX therefore must find a contingent strategy, which specifies MAX's move in the initial state.
  - Then, MAX's moves in the states resulting from every possible response by MIN, then MAX's moves in the states resulting from every possible response by MIN to those moves, and so on.

Optimal Decisions in Games

- Given a game tree, the **optimal strategy** can be determined from the minimax value of each node, which we write as MINIMAX(n).

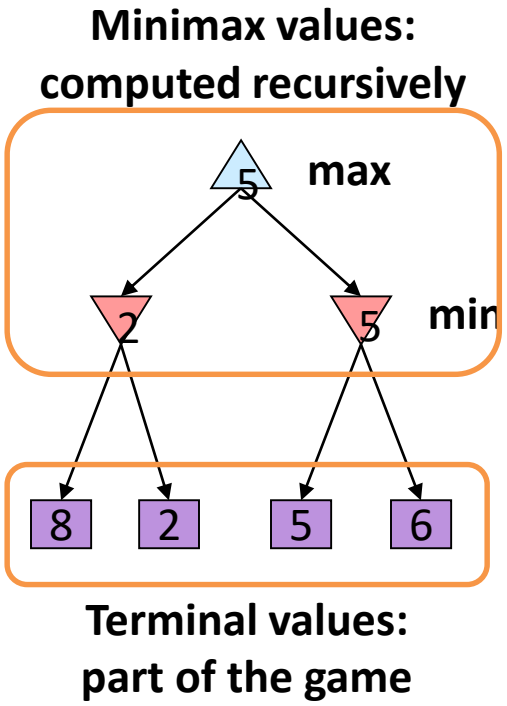
$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

- The minimax value of a node is the utility (for MAX) of being in the corresponding state, assuming that both players play optimally from there to the end of the game.
  - The minimax value of a terminal state is just its utility.
  - MAX prefers to move to a state of maximum value.
  - MIN prefers a state of minimum value.

Adversarial Search (Minimax)

- Deterministic, zero-sum games:
  - Tic-tac-toe, chess, checkers
  - One player maximizes result
  - The other minimizes result

- Minimax search:
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary



Optimal Decisions in Games

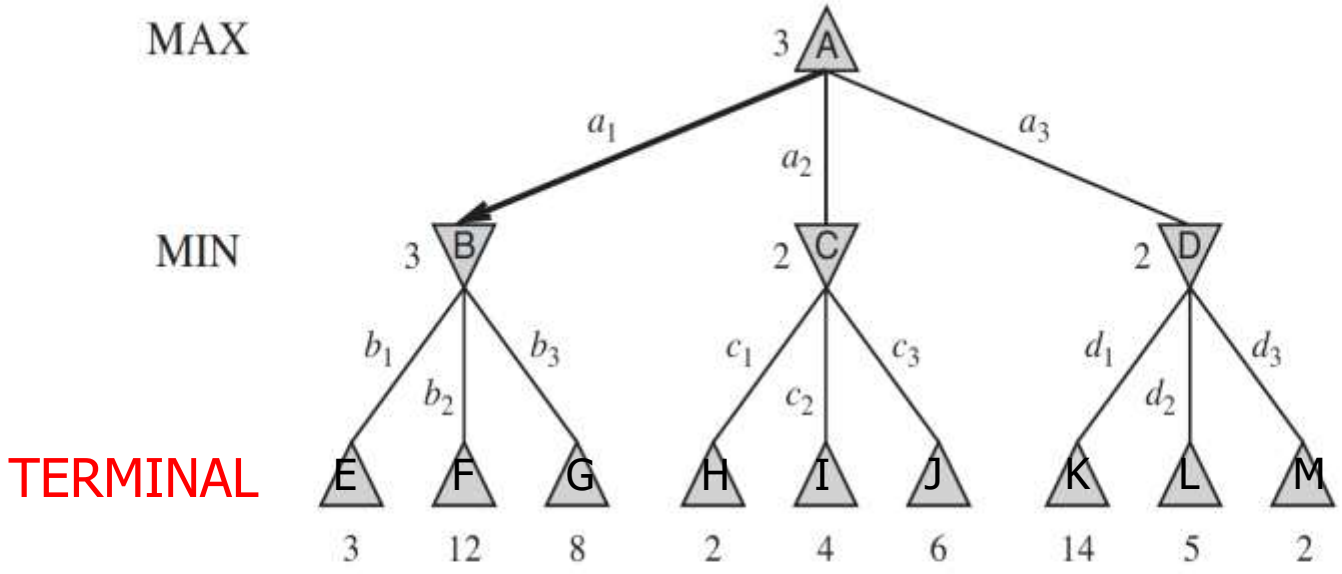
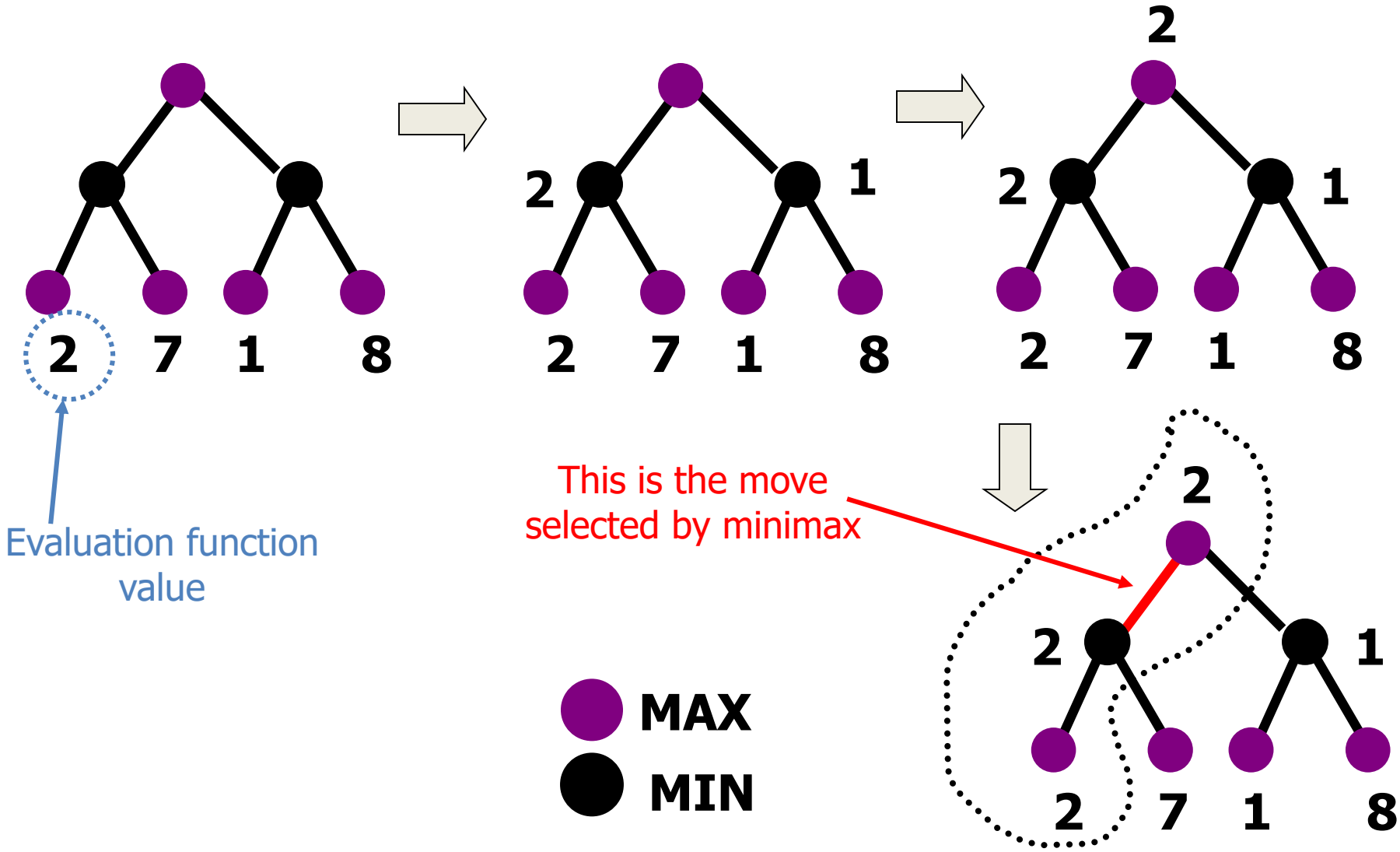


Figure 5.2 A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN's best reply is b_1, because it leads to the state with the lowest minimax value.

Minimax Example



What if MIN does not play optimally?

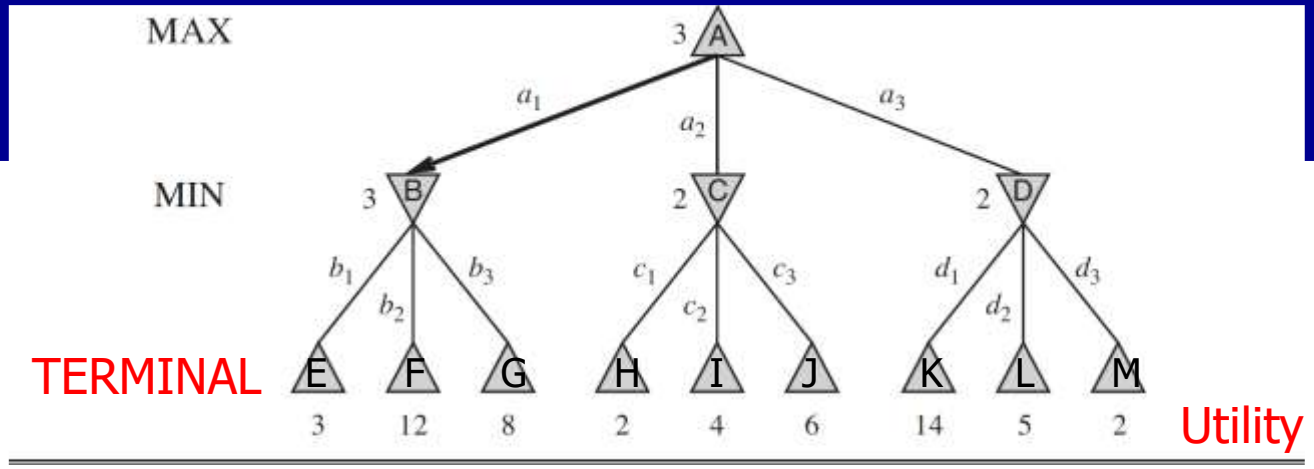
- Definition of optimal play for MAX assumes MIN plays optimally:
  - *Maximizes worst-case outcome* for MAX.

- But if MIN does not play optimally, MAX will do even better.
  - Theorem-not hard to prove

Minimax Algorithm

- The minimax algorithm computes the minimax decision from the current state.
- It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.

Minimax Algorithm



function MINIMAX-DECISION(*state*) **returns** *an action*
  inputs: *state*, current state in game
  v←MAX-VALUE(*state*)
  return the *action* in SUCCESSORS(*state*) with value *v*

---

function MAX-VALUE(*state*) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  v ← -∞
  for *a,s* in SUCCESSORS(*state*) **do**
    v ← MAX(*v*, *MIN-VALUE(s)*)
  return *v*

---

function MIN-VALUE(*state*) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  v ← ∞
  for *a,s* in SUCCESSORS(*state*) **do**
    v ← MIN(*v*, *MAX-VALUE(s)*)
  return *v*

Minimax Implementation (Dispatch)

```
def value(state):
        if the state is a terminal state: return the state's utility
        if the next agent is MAX: return max-value(state)
        if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

Comments on Minimax Search

- The minimax algorithm performs a complete depth-first exploration of the game tree.

- Depth-first search with fixed number of ply as the limit.
  - $O(b^m)$ time complexity – *Ooops*!
  - O(bm) space complexity

- The time cost is totally impractical, but this algorithm serves as the basis for the mathematical analysis of games and for more practical algorithms.

Comments on Minimax Search

- Performance will depend on
  - the quality of the evaluation function (expert knowledge)
  - depth of search (computing power and search algorithm)

- Differences from normal state space search
  - Looking for one move only
  - No cost on arcs
  - MAX can't be sure how MIN will respond to his moves

- Minimax rule forms the basis for other game tree search algorithms.

Search tree size !!

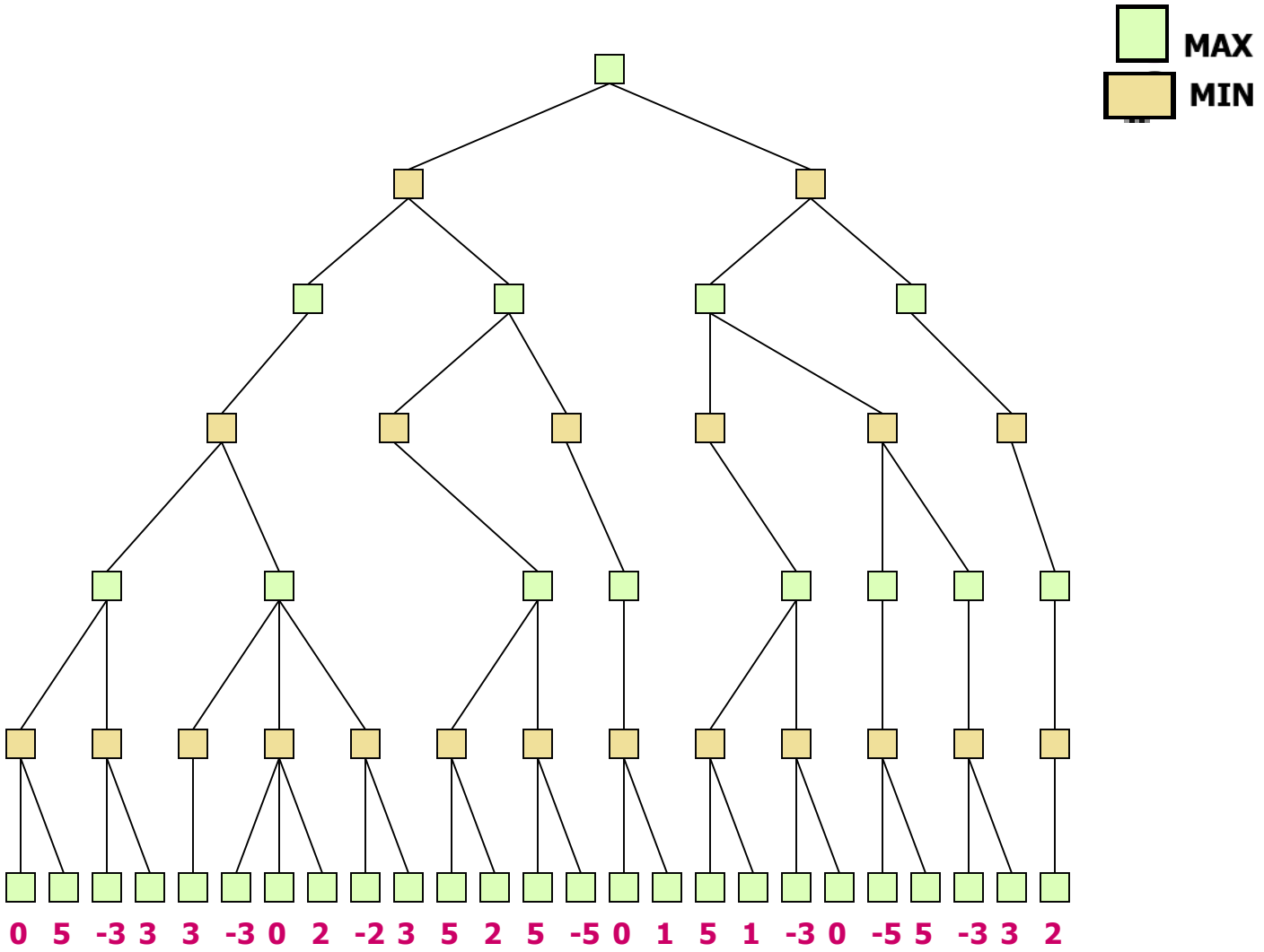
But in general, the search tree is too big to make it possible to reach the terminal states!

Examples:

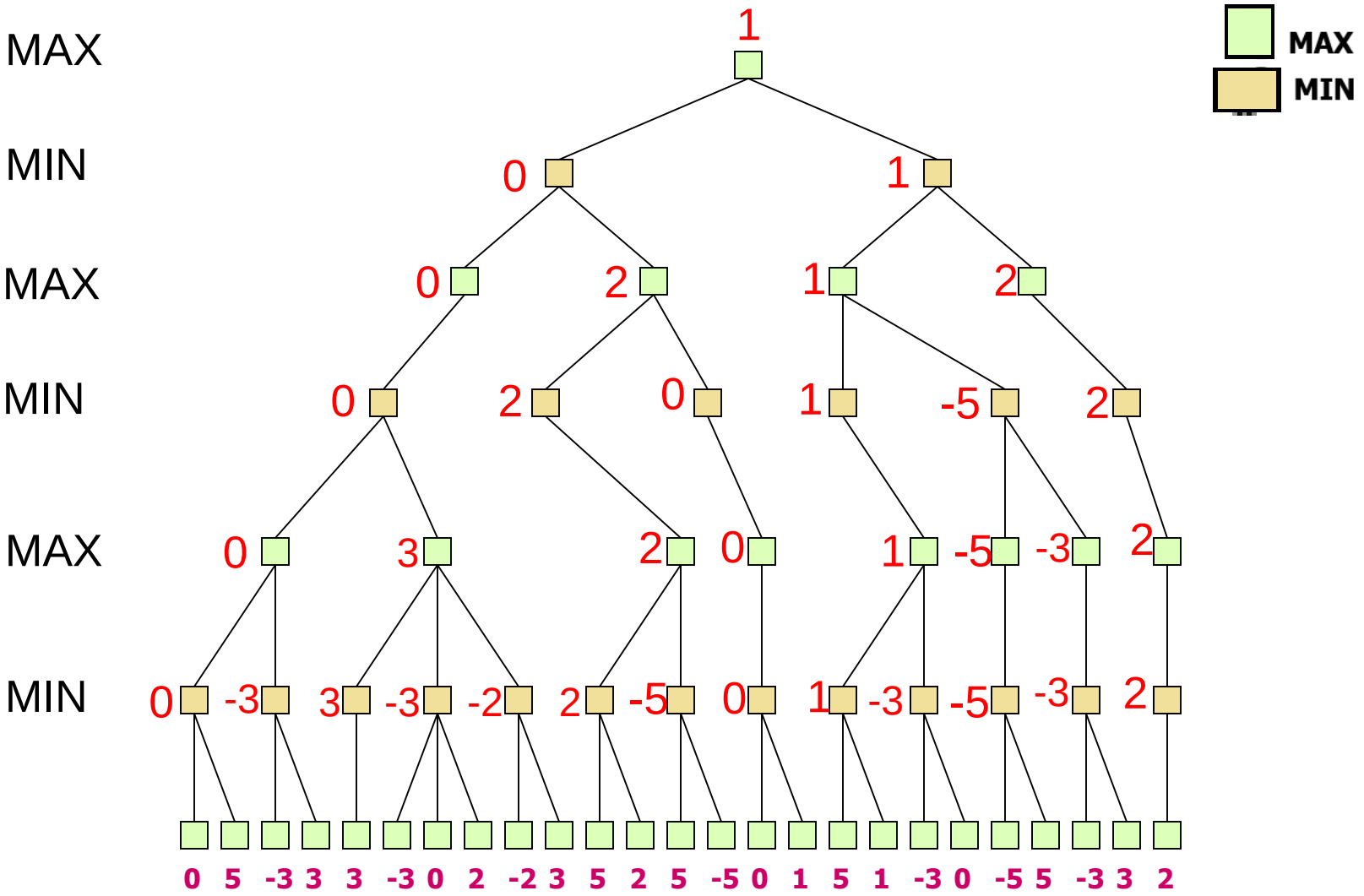
- Checkers: ~10^{40} nodes
- Chess: ~10^{120} nodes

Exact solution is completely infeasible.
But, do we need to explore the whole tree?

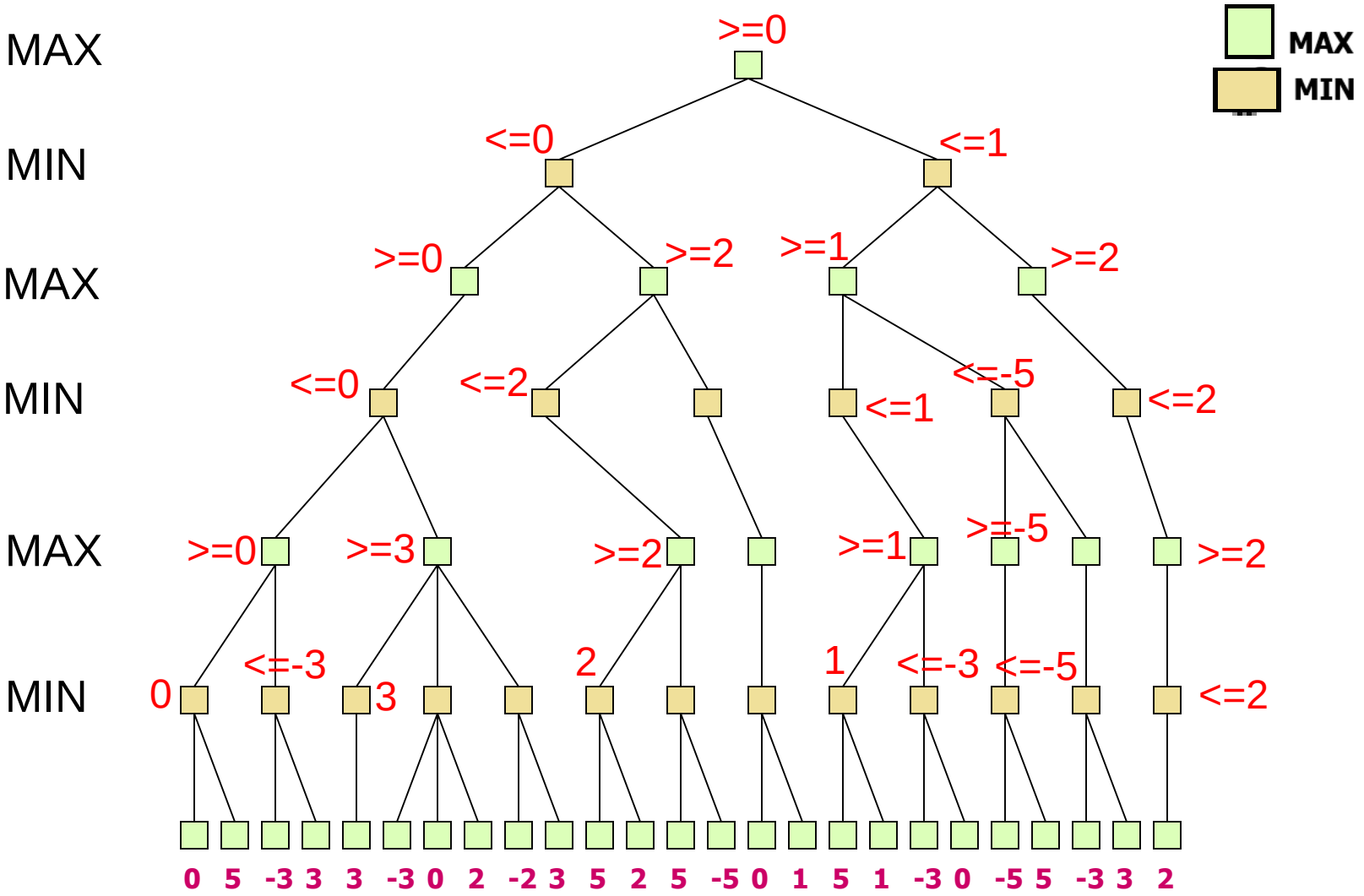
Minimax Example



Minimax Example



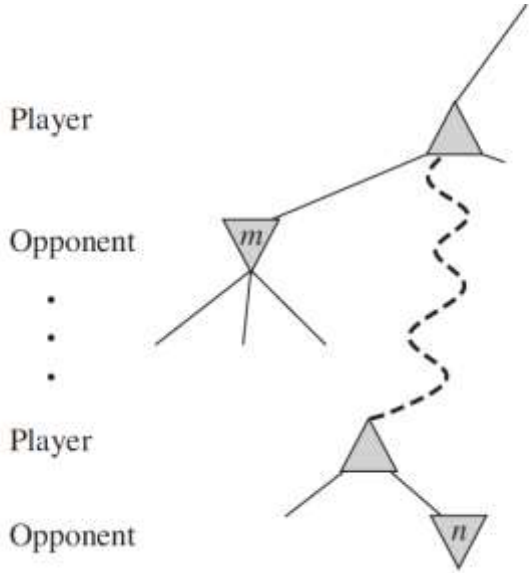
Alpha-Beta Pruning



Alpha-Beta Pruning

- A way to improve the performance of the Minimax Procedure

If m is better than n for Player, we will never get to n in play.

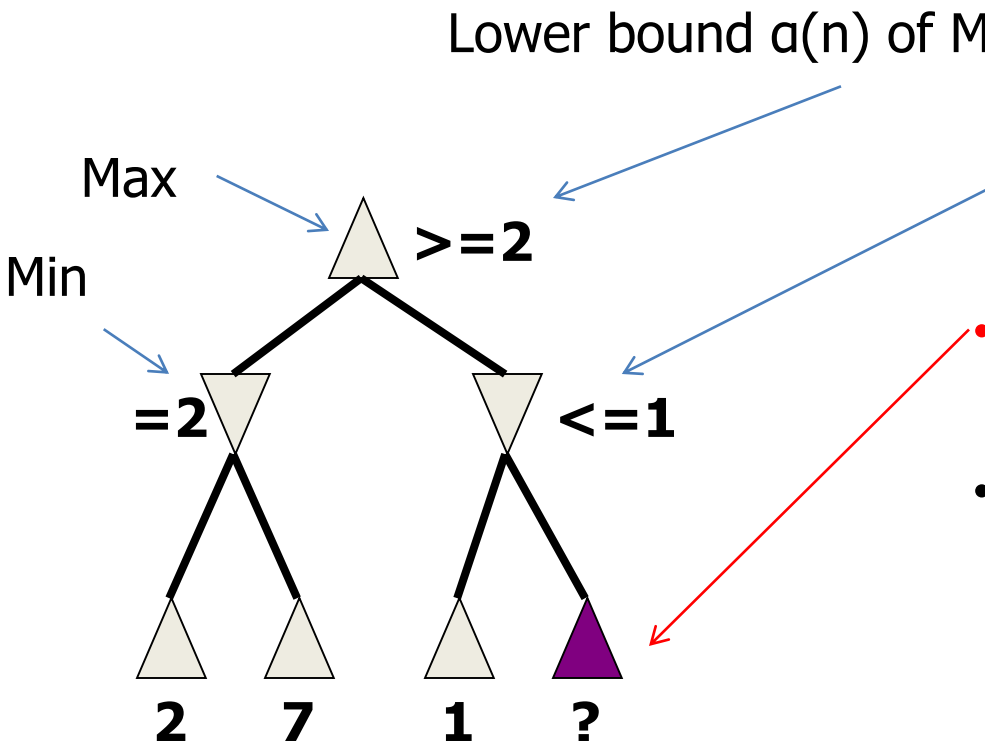


- α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

- β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

Alpha-Beta Pruning

- A way to improve the performance of the Minimax Procedure

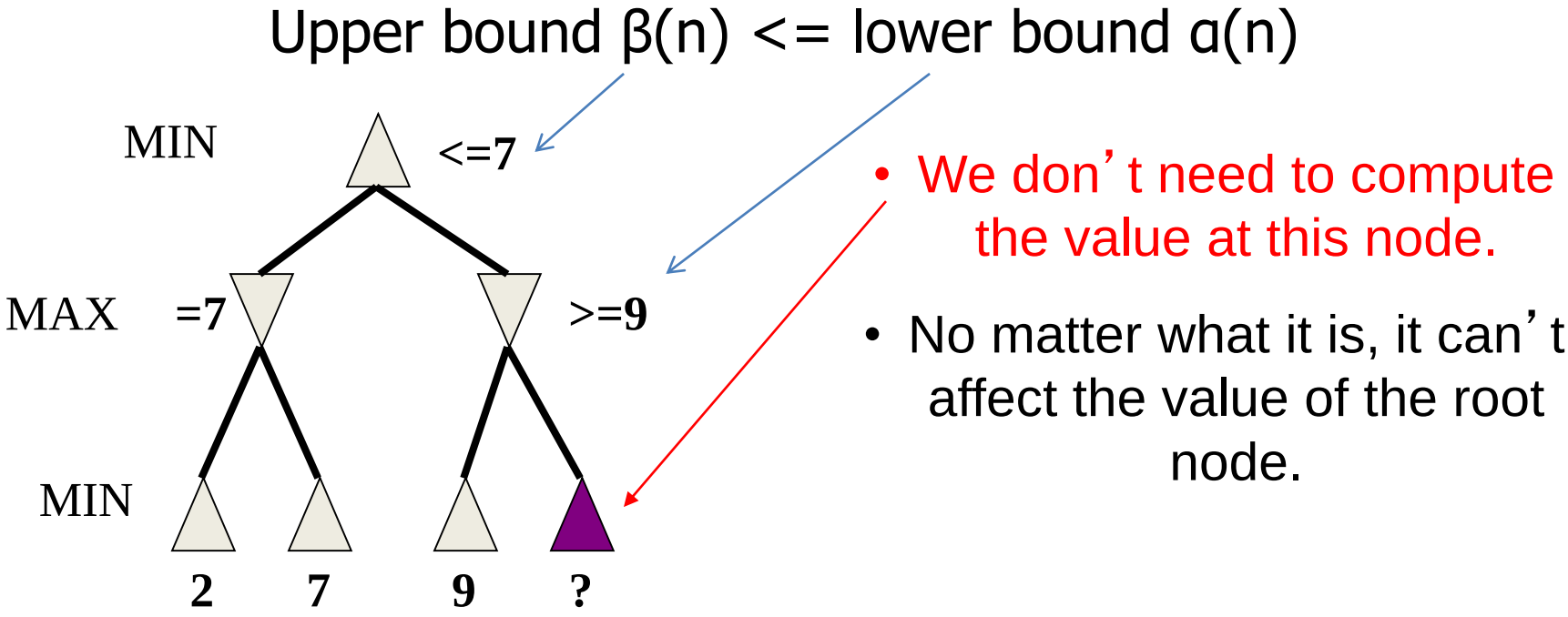


Upper bound β(n) of MIN node n

- We don't need to compute the value at this node.
- No matter what it is, it can't affect the value of the root node.

Alpha-Beta Pruning

- We can improve on the performance of the minimax algorithm through alpha-beta pruning .



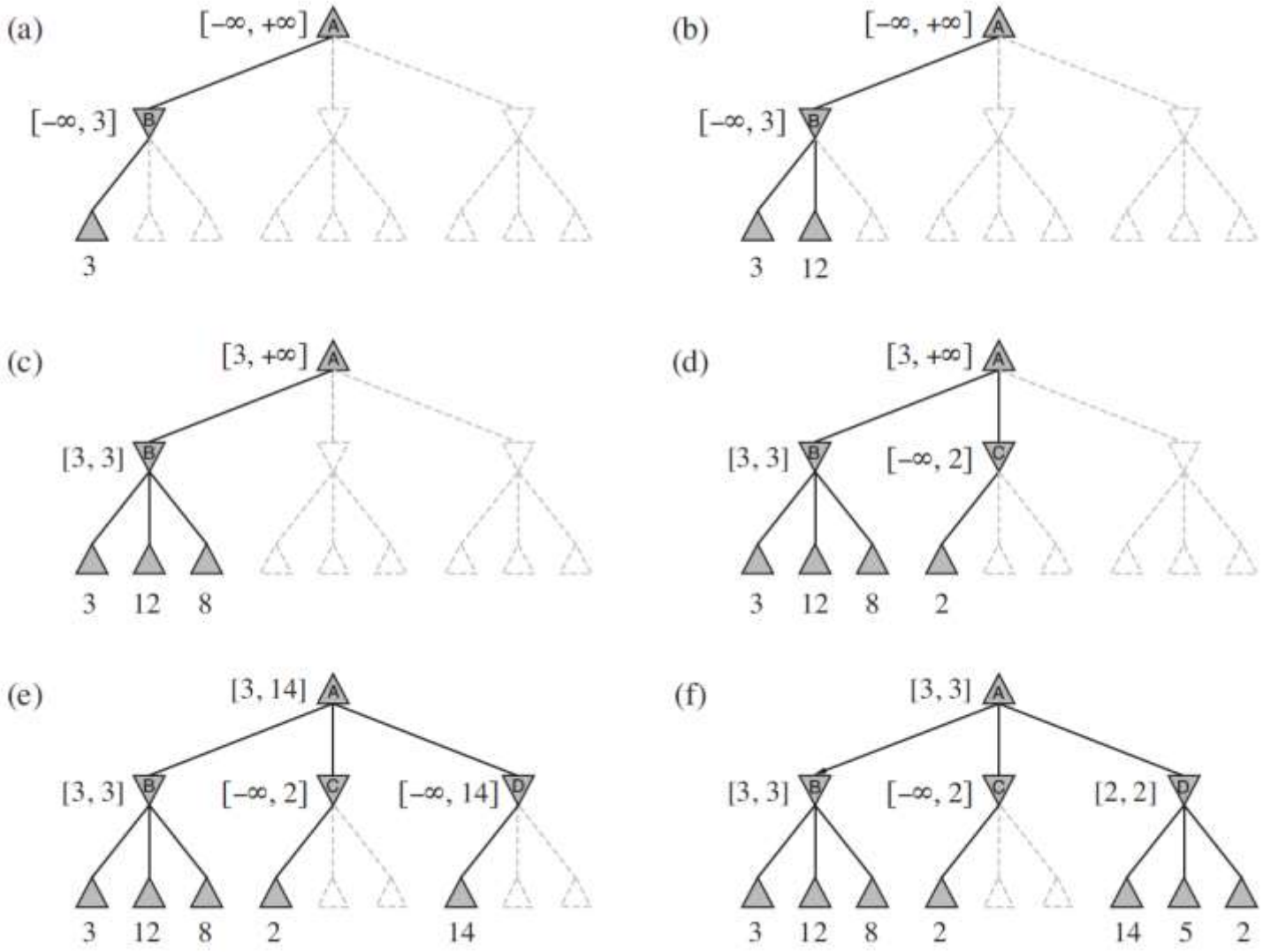
Alpha-Beta Pruning

- Traverse the search tree in depth-first order
- For each MAX node n, α(n)=maximum child value found so far
  - Starts with – ∞
  - Increases if a child returns a value greater than the current α(n)
  - **Lower-bound** on the final value

- For each MIN node n, β(n)=minimum child value found so far
  - Starts with + ∞
  - Decreases if a child returns a value less than the current β(n)
  - **Upper-bound** on the final value

- MAX cutoff rule: at a MAX node n, cut off search if α(n)>=β(n)
- MIN cutoff rule: at a MIN node n, cut off search if β(n)<=α(n)
- Carry α and β values down in search

Alpha-Beta Pruning



Alpha-Beta Algorithm

function ALPHA-BETA-SEARCH(*state*) **returns** *an action*
inputs: *state*, current state in game
$v \leftarrow$ MAX-VALUE(*state*, $-\infty$, $+\infty$)
return the *action* in SUCCESSORS(*state*) with value *v*

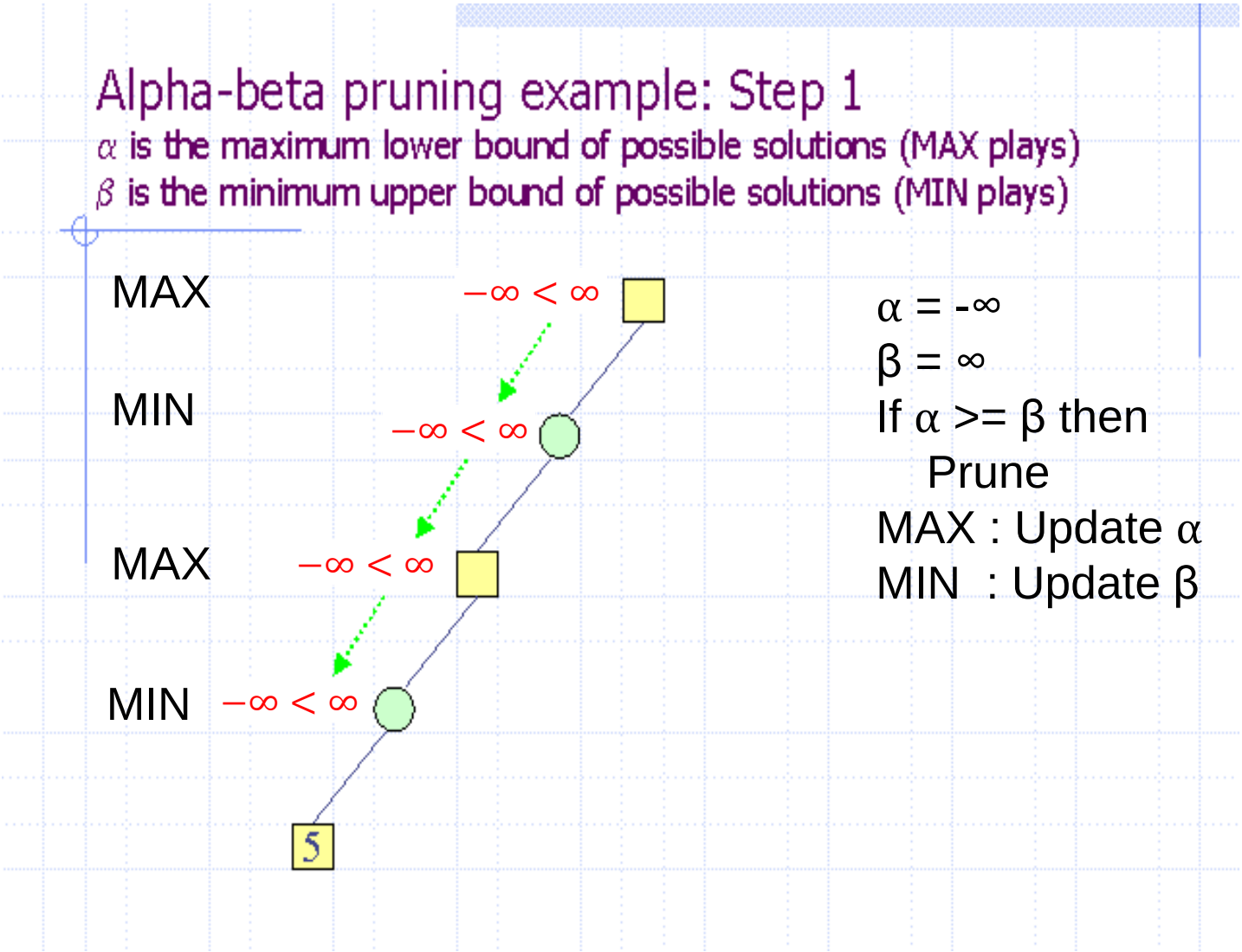
---

function MAX-VALUE(*state*, α , β) **returns** *a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
$v \leftarrow -\infty$
for *a,s* in SUCCESSORS(*state*) **do**
$v \leftarrow$ MAX(v,MIN-VALUE(s, α , β))
if $v \geq \beta$ **then return** v
$\alpha \leftarrow$ MAX(α ,v)
return v

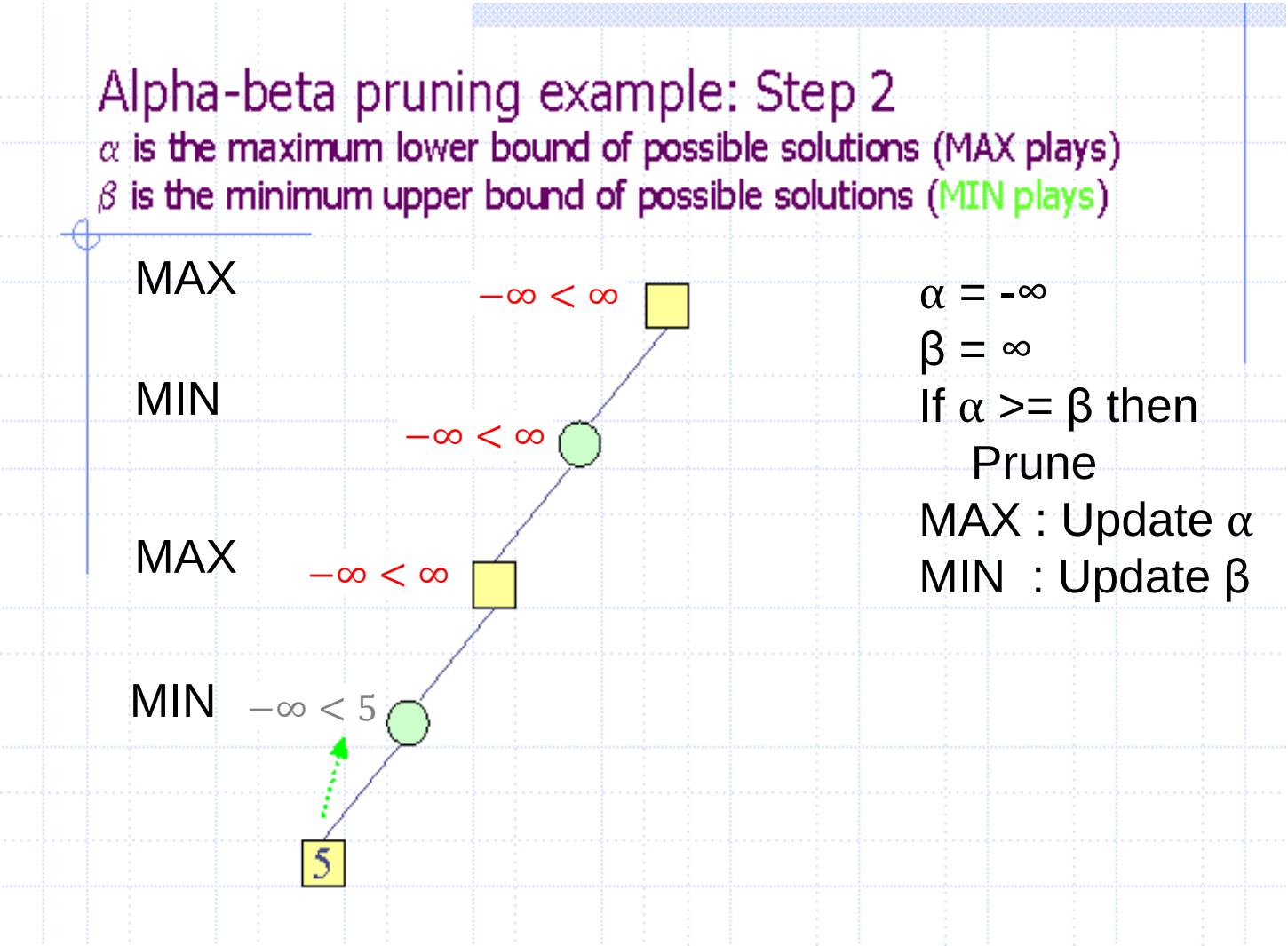
---

function MIN-VALUE(*state*, α , β) **returns** *a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
$v \leftarrow +\infty$
for *a,s* in SUCCESSORS(*state*) **do**
$v \leftarrow$ MIN(v,MAX-VALUE(s, α , β))
if $v \leq \alpha$ **then return** v
$\beta \leftarrow$ MIN(β ,v)
return v

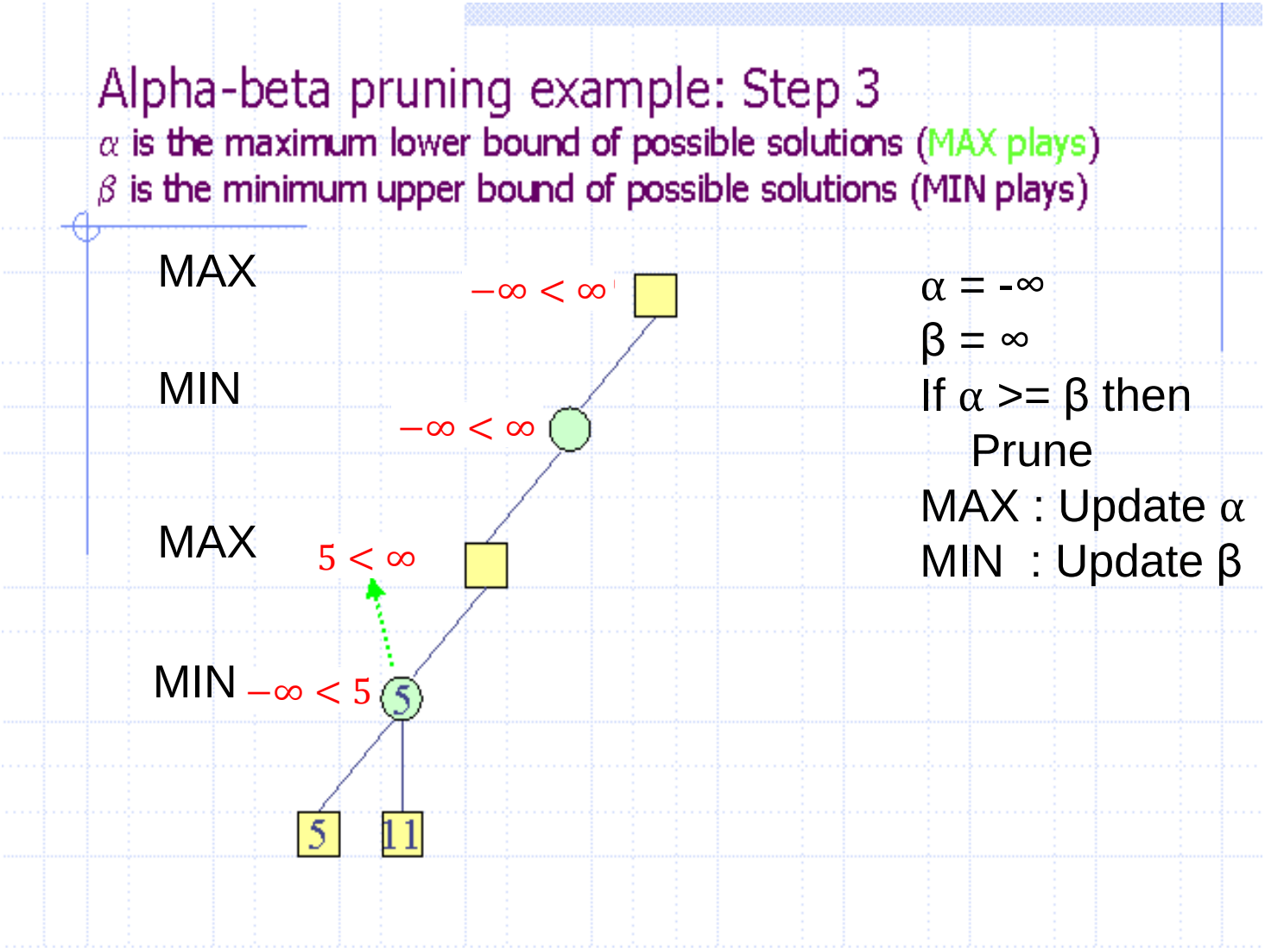
Alpha-Beta Pruning



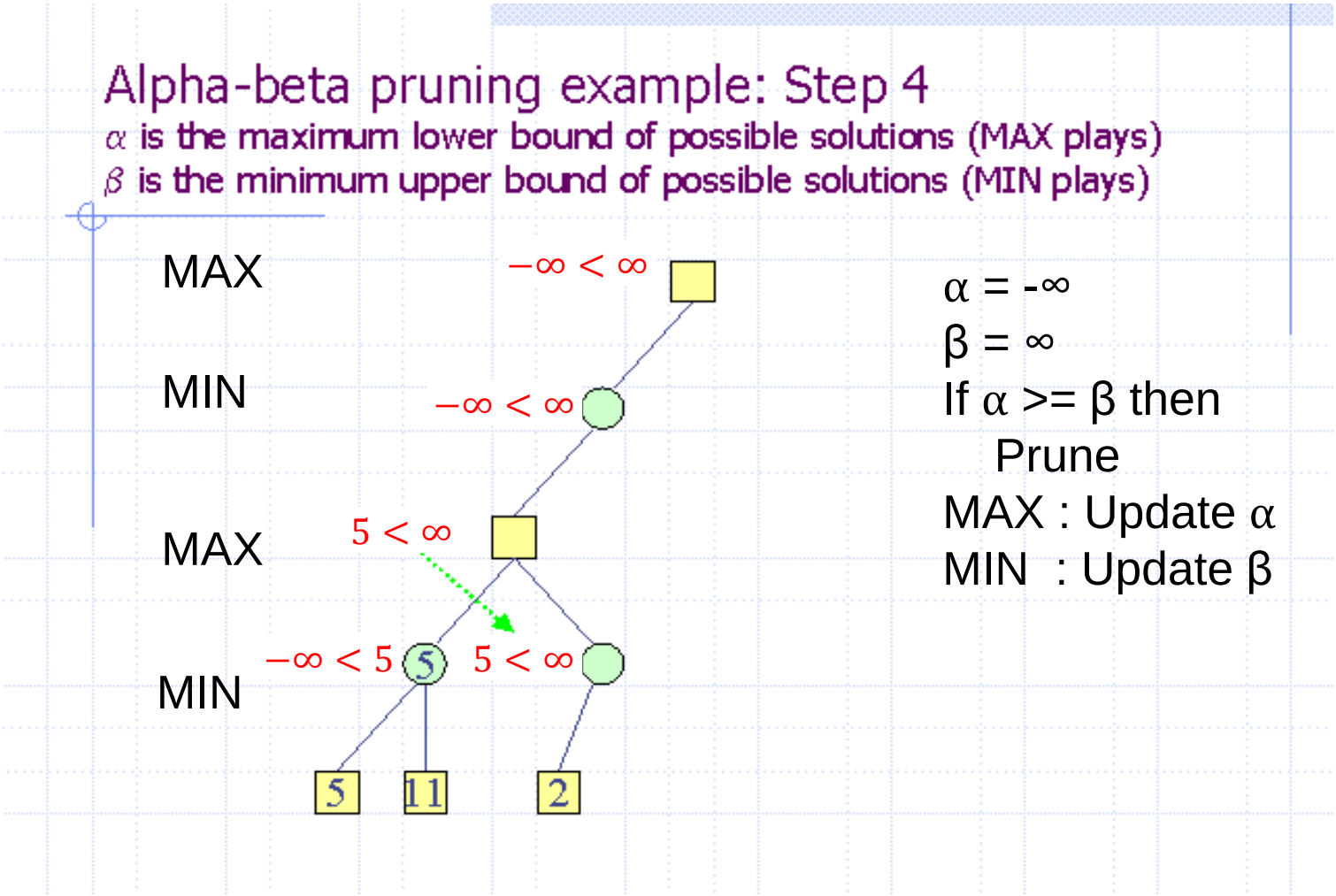
Alpha-Beta Pruning (cont.)



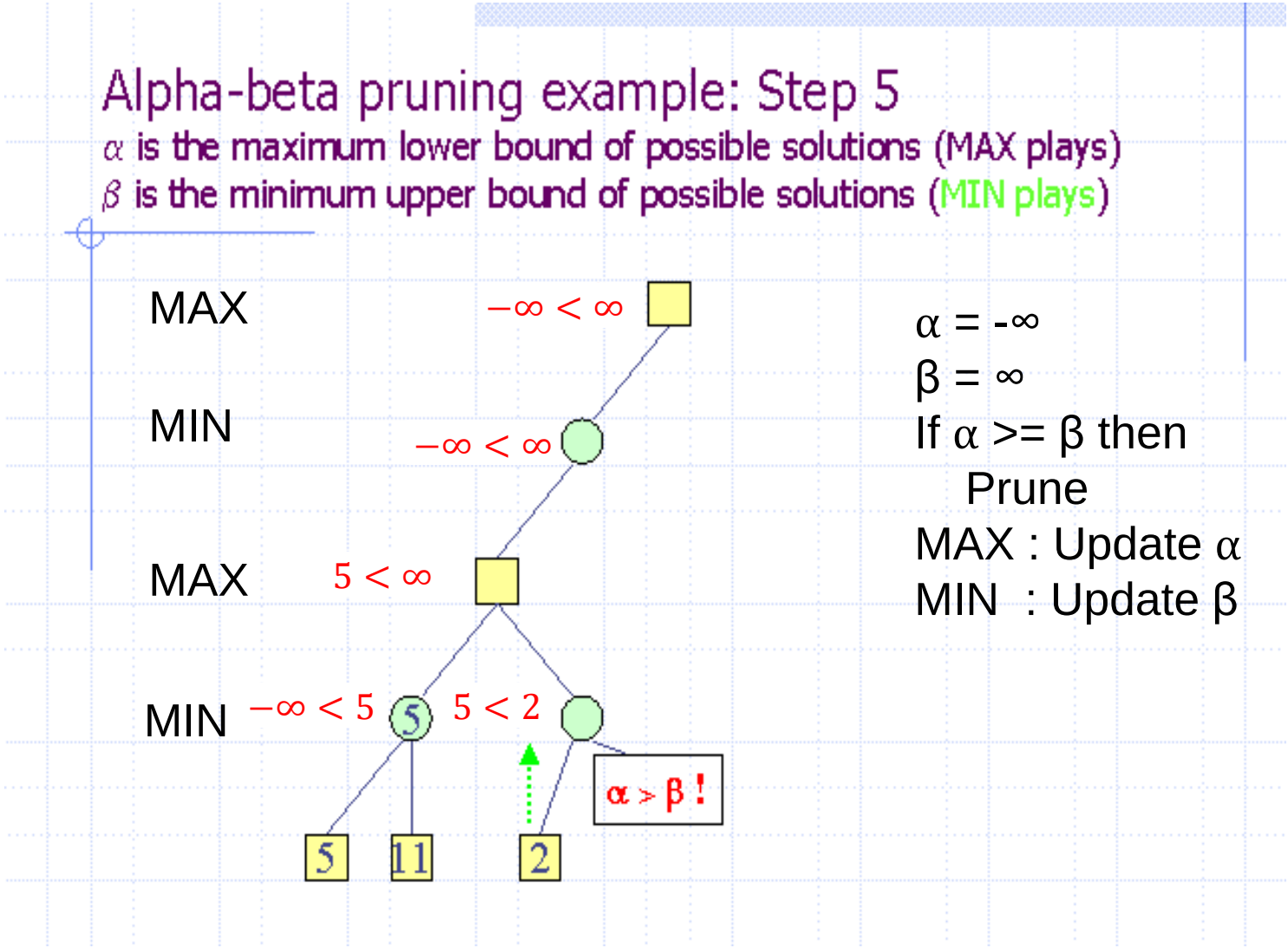
Alpha-Beta Pruning (cont.)



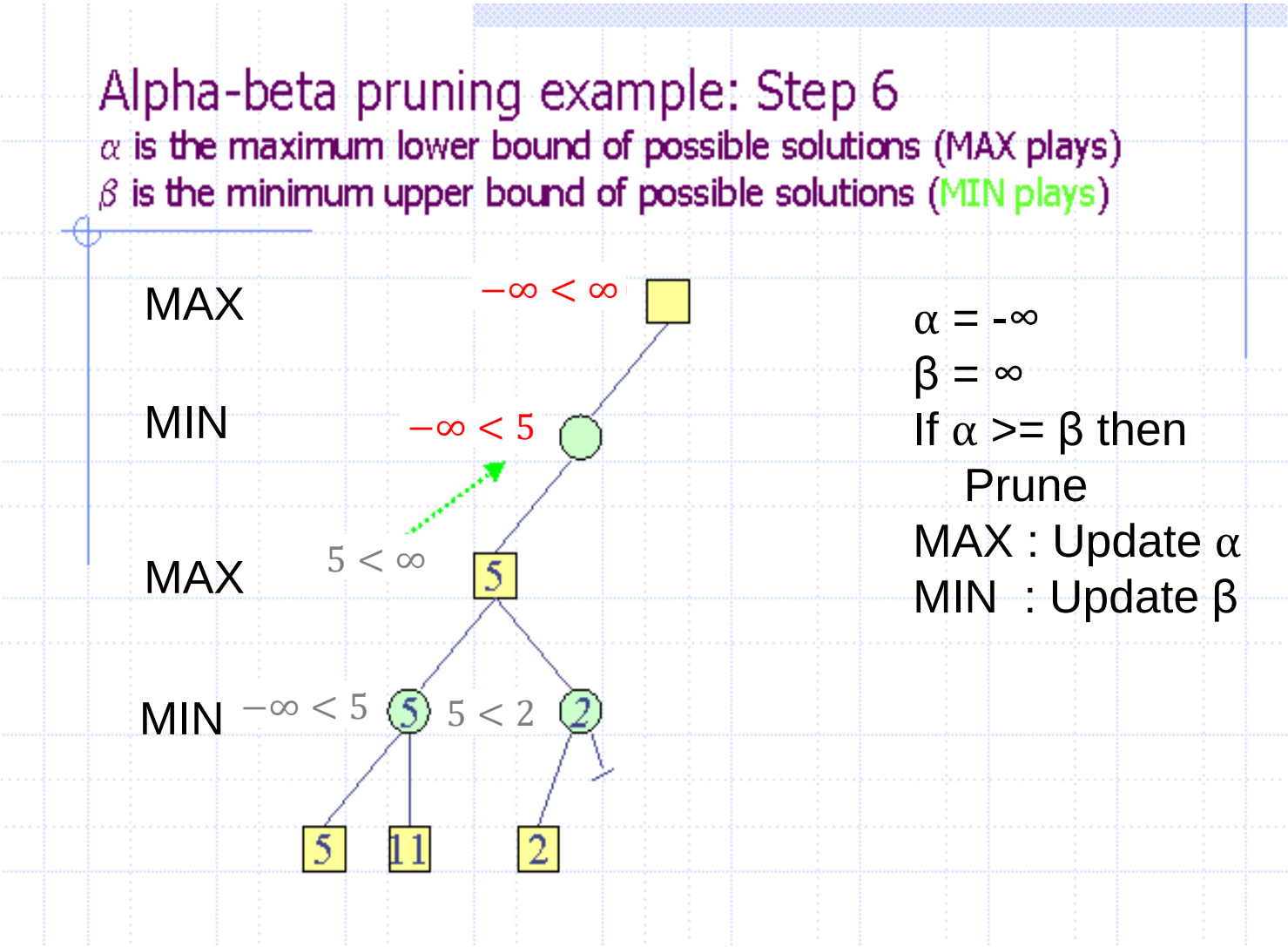
Alpha-Beta Pruning (cont.)



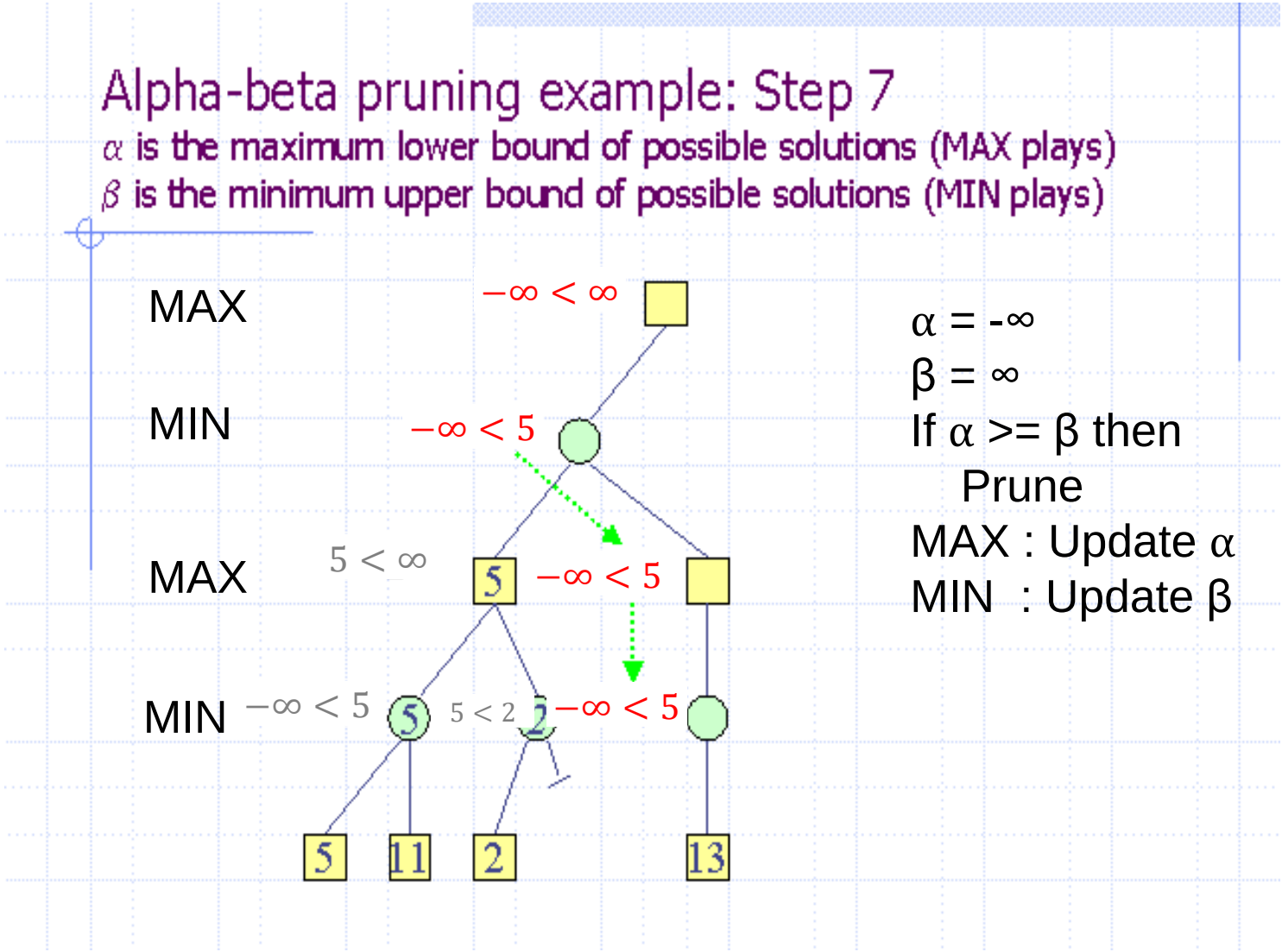
Alpha-Beta Pruning (cont.)



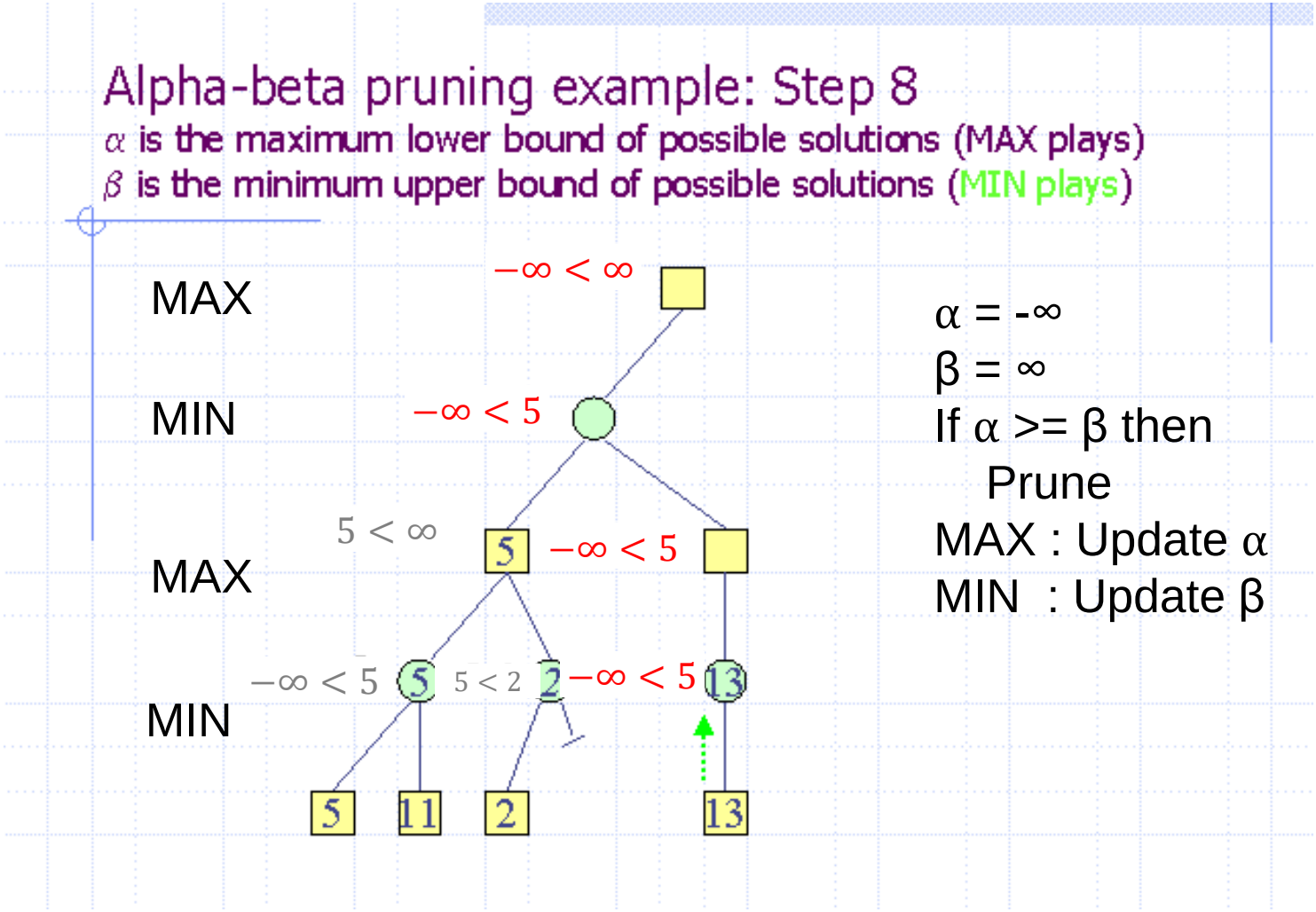
Alpha-Beta Pruning (cont.)



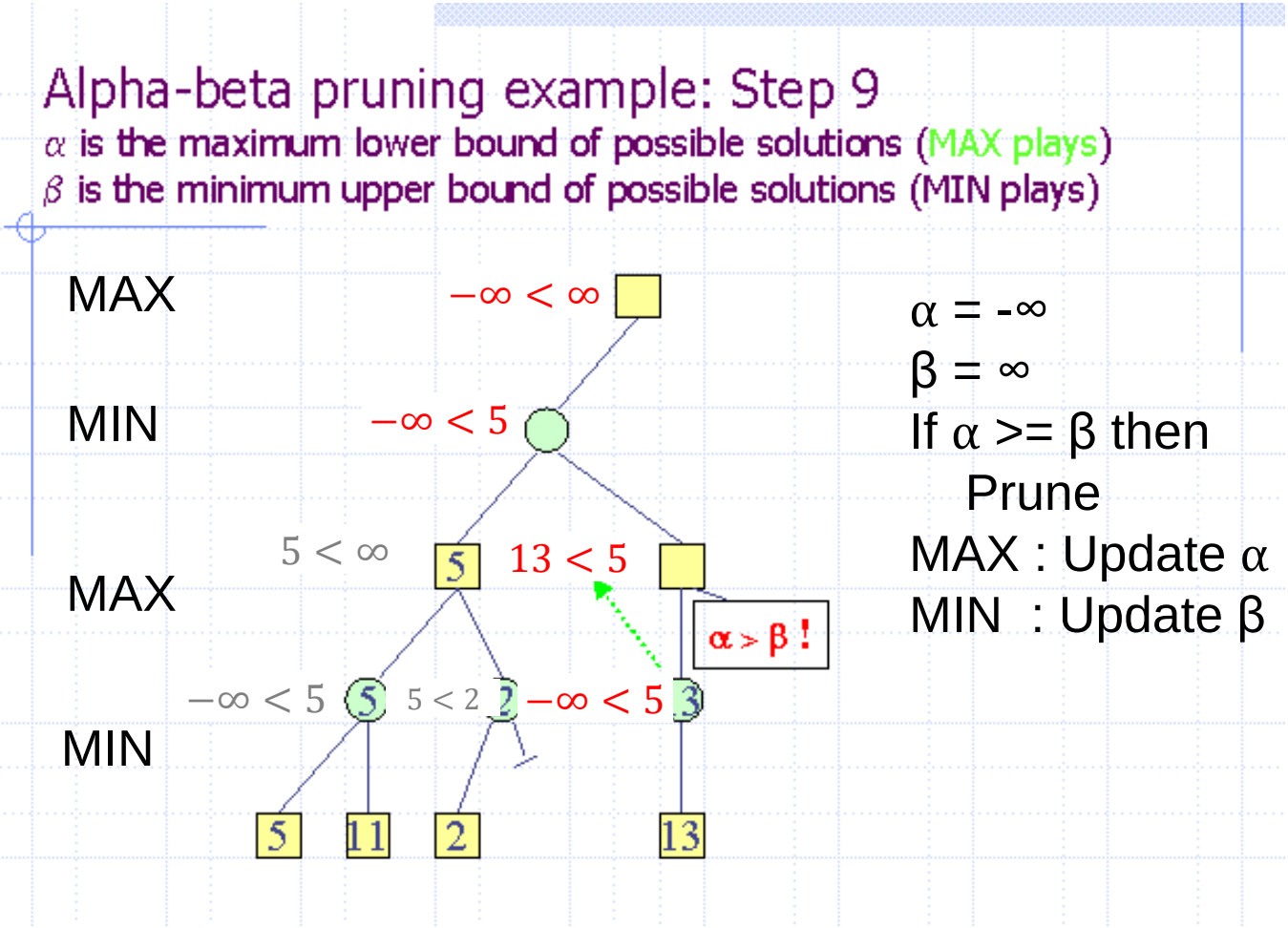
Alpha-Beta Pruning (cont.)



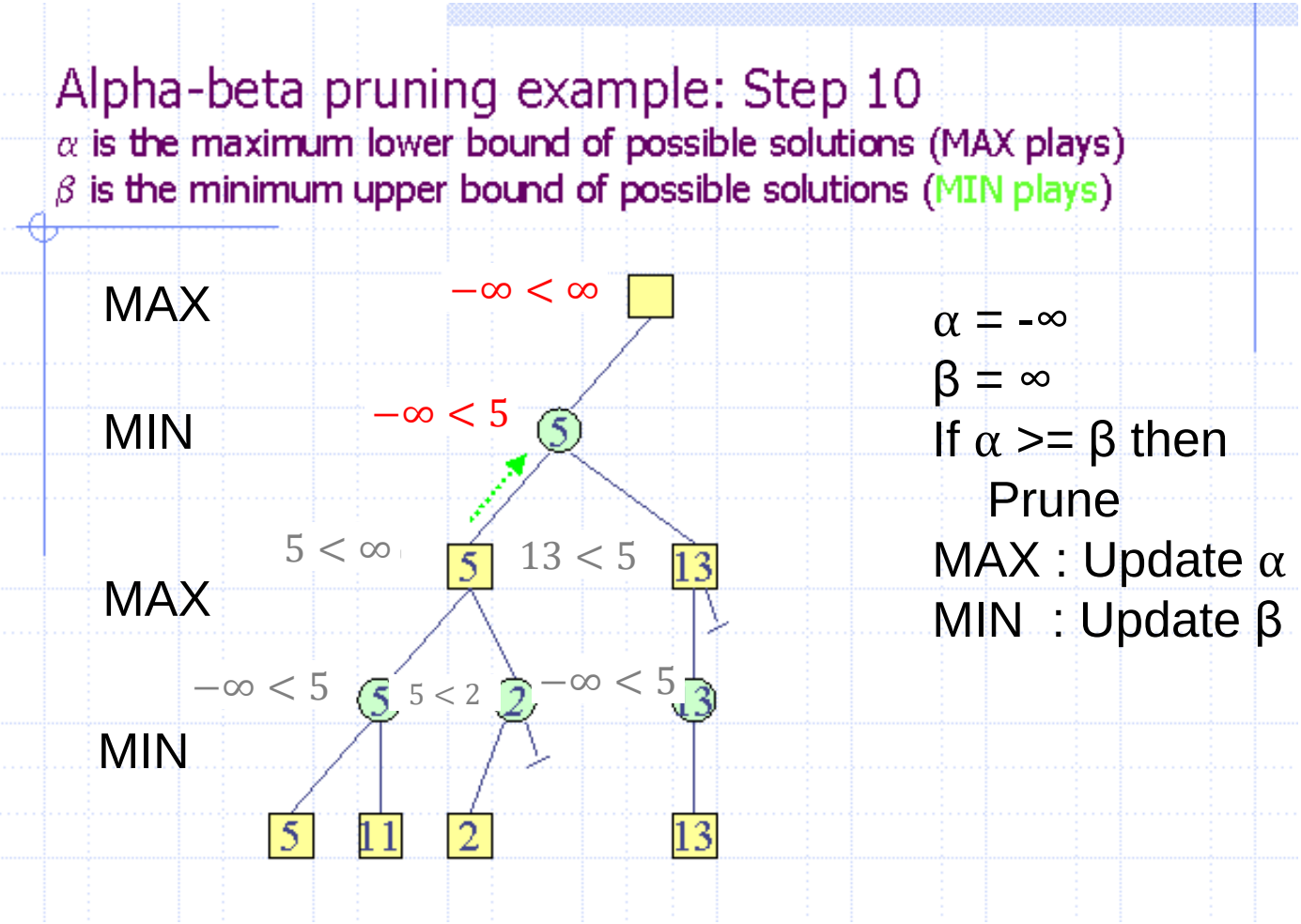
Alpha-Beta Pruning (cont.)



Alpha-Beta Pruning (cont.)

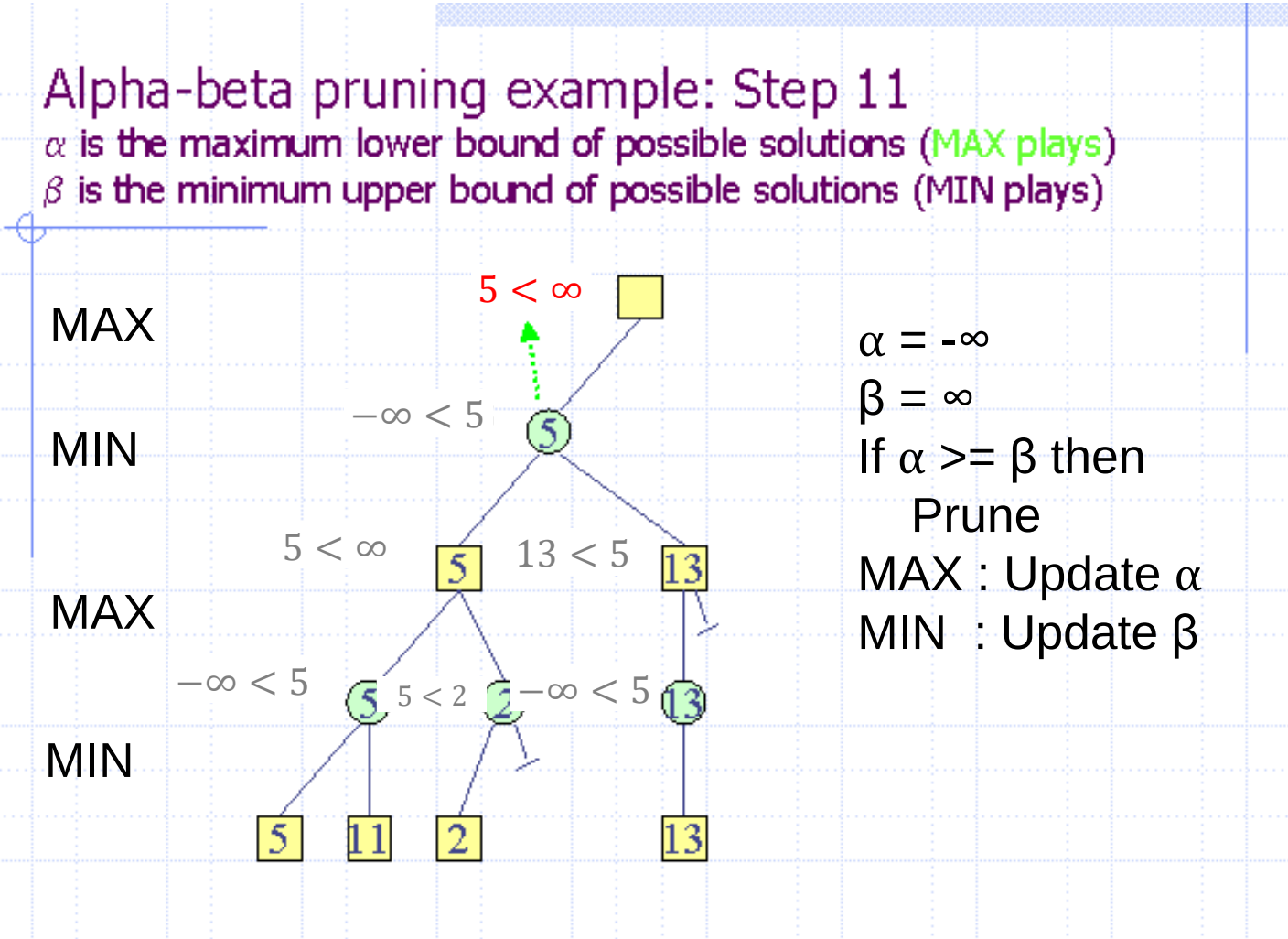


Alpha-Beta Pruning (cont.)

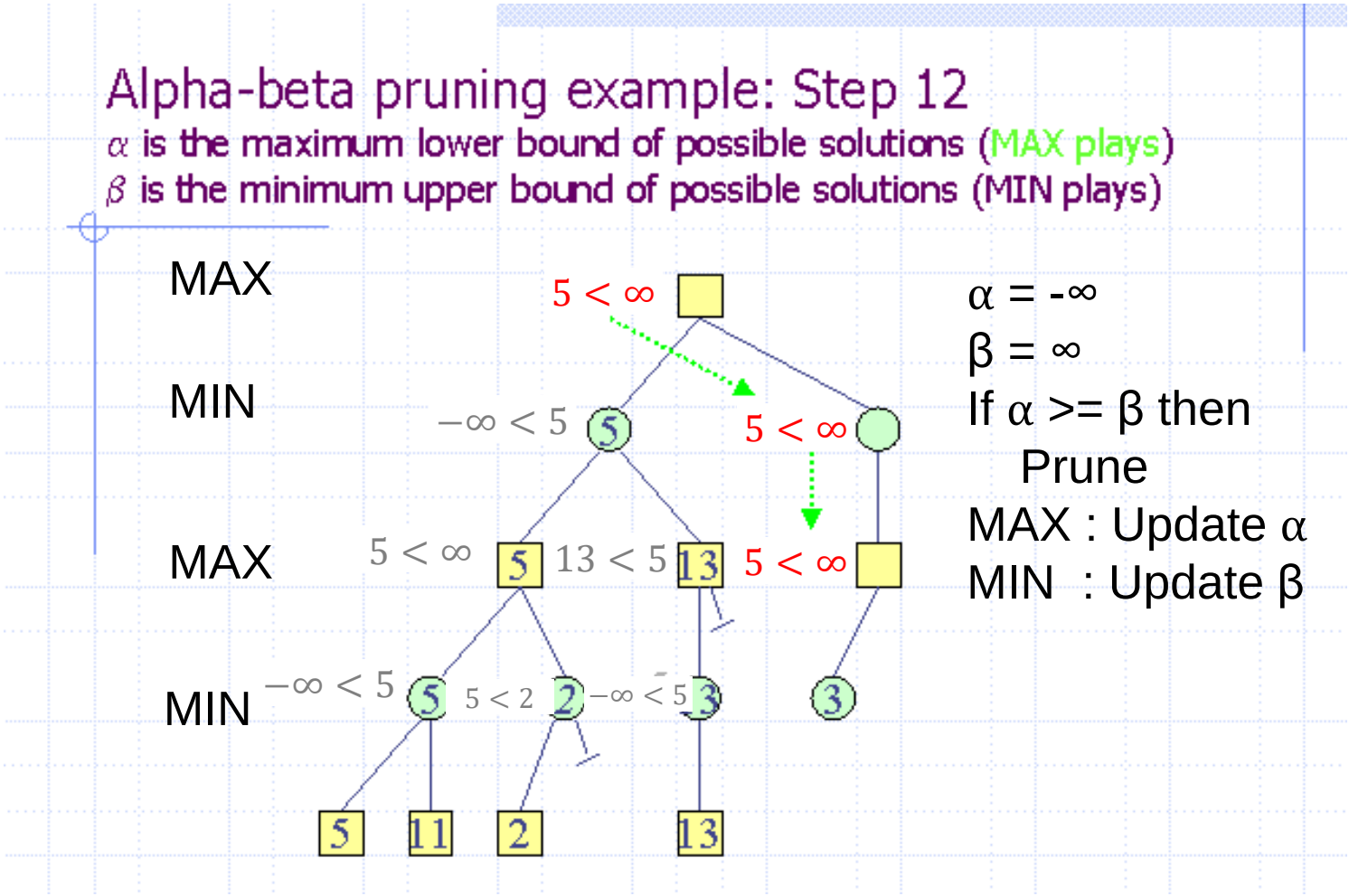


α = -∞
β = ∞
If α >= β then
Prune
MAX : Update α
MIN : Update β

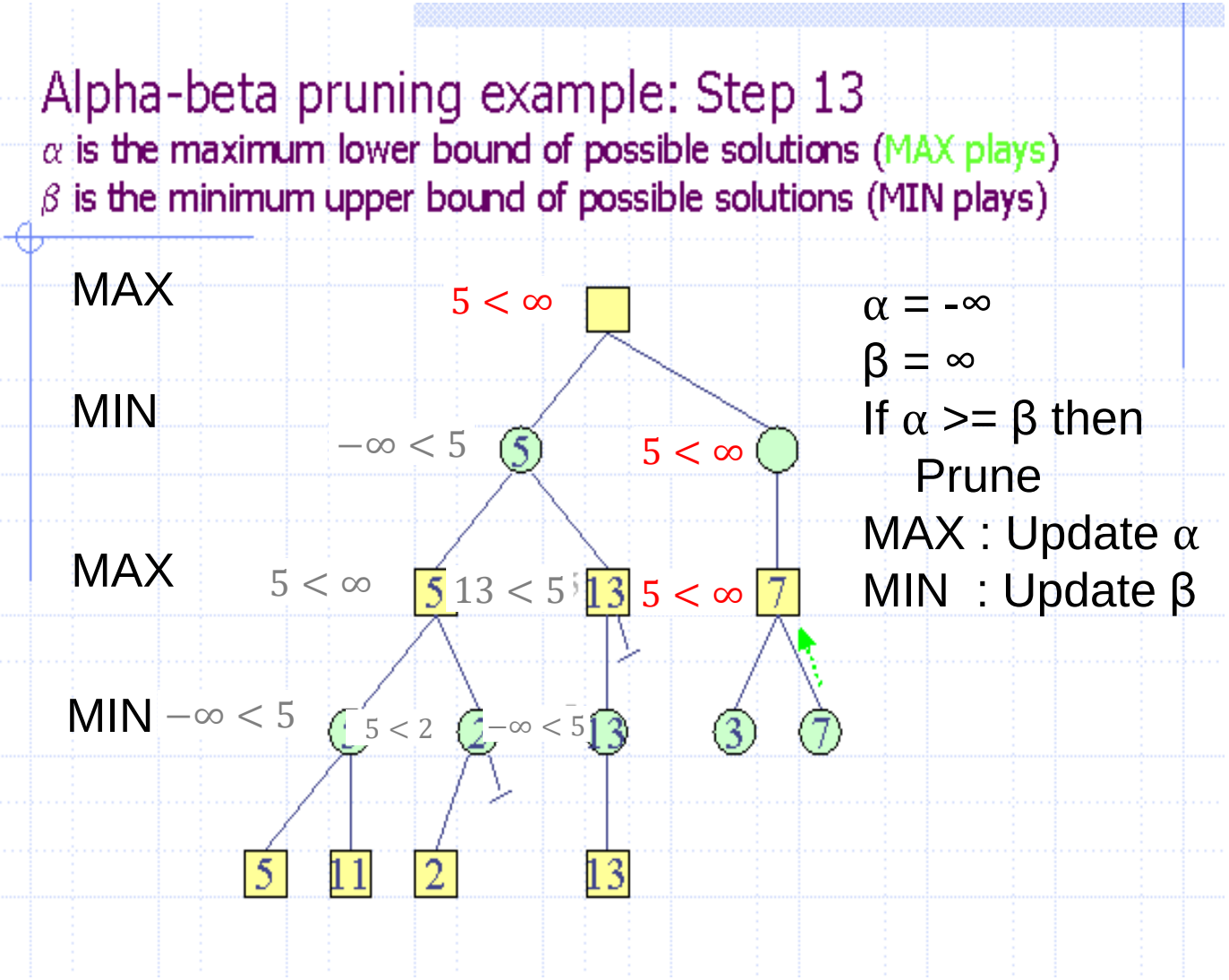
Alpha-Beta Pruning (cont.)



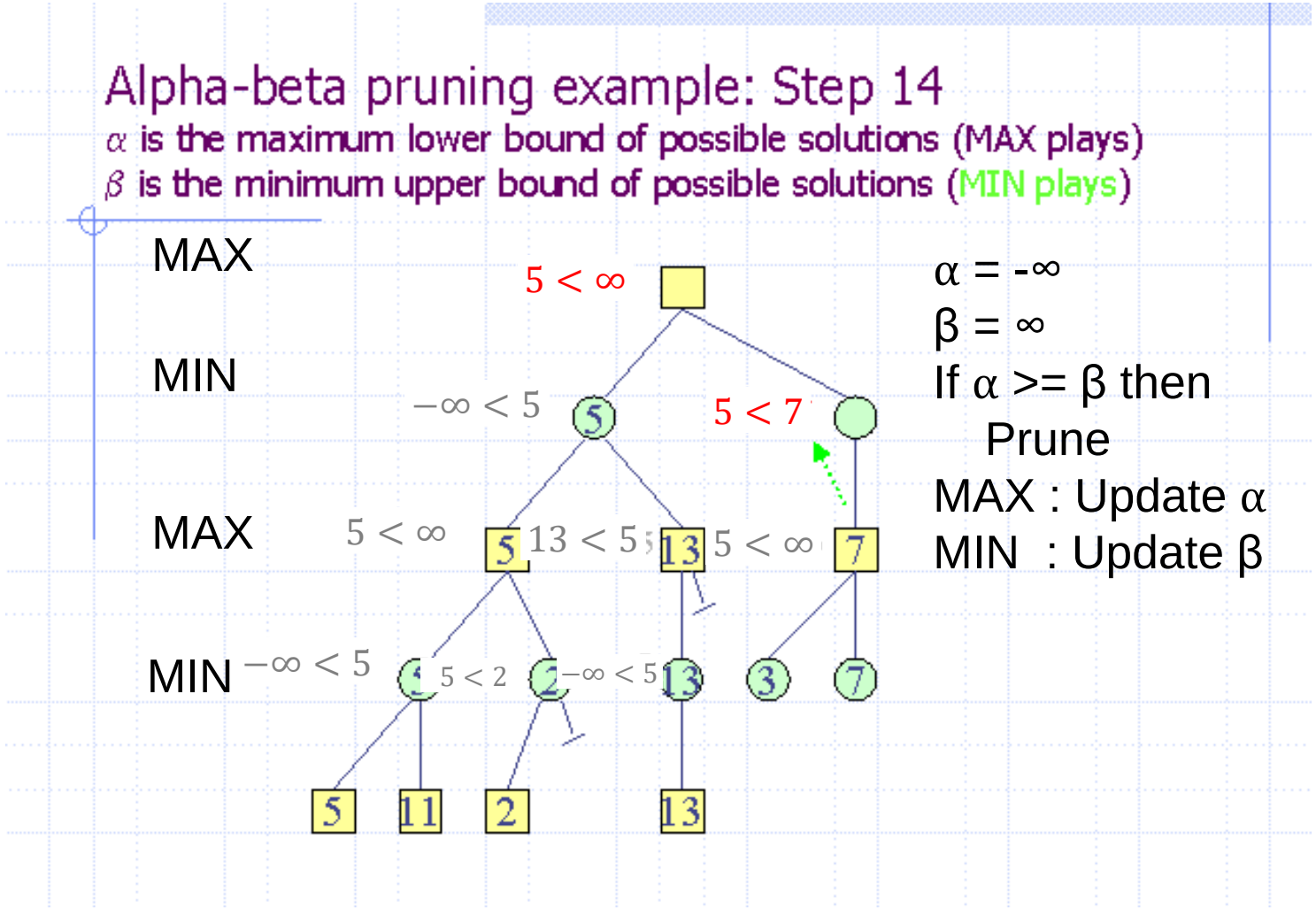
Alpha-Beta Pruning (cont.)



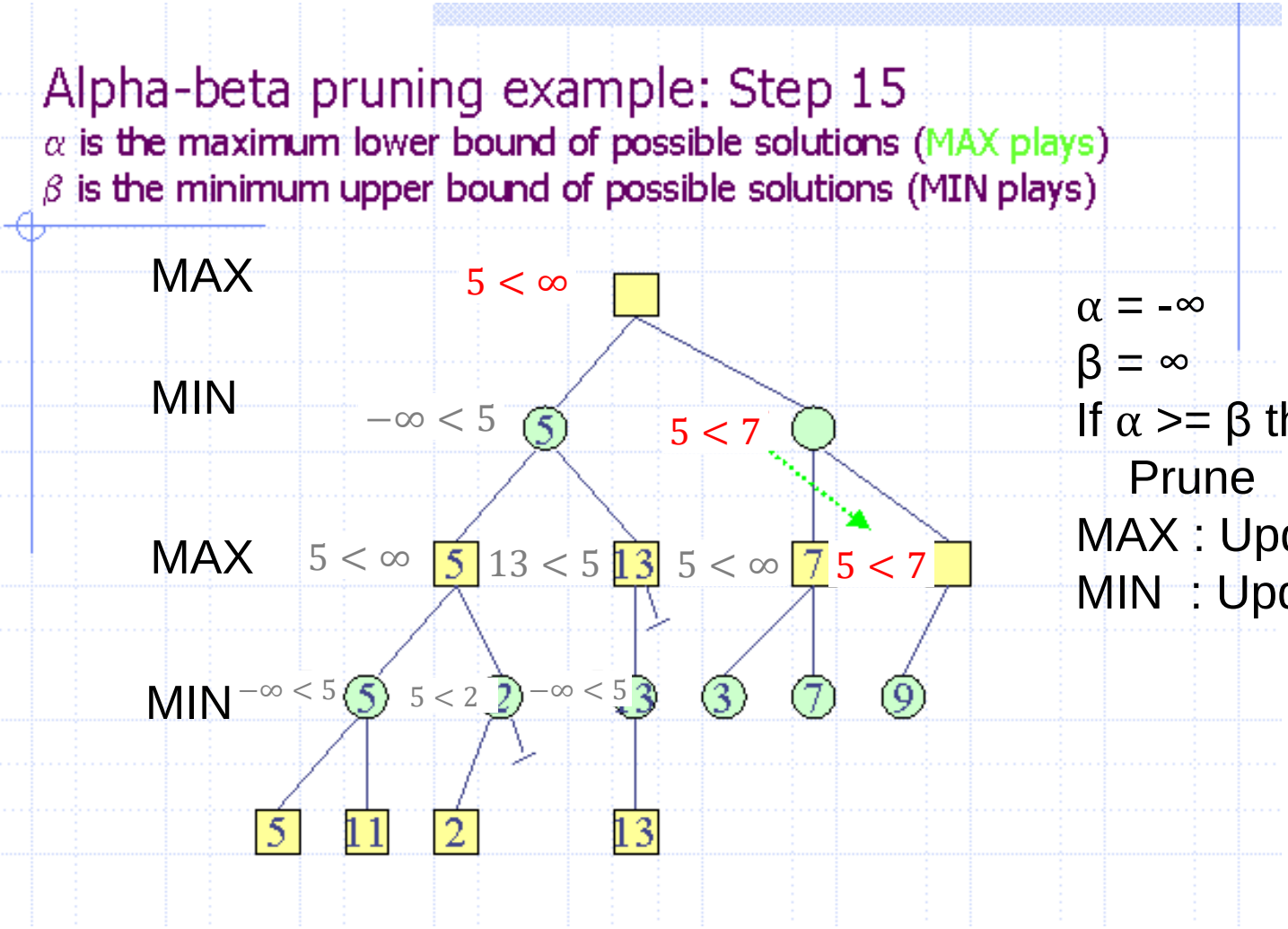
Alpha-Beta Pruning (cont.)



Alpha-Beta Pruning (cont.)

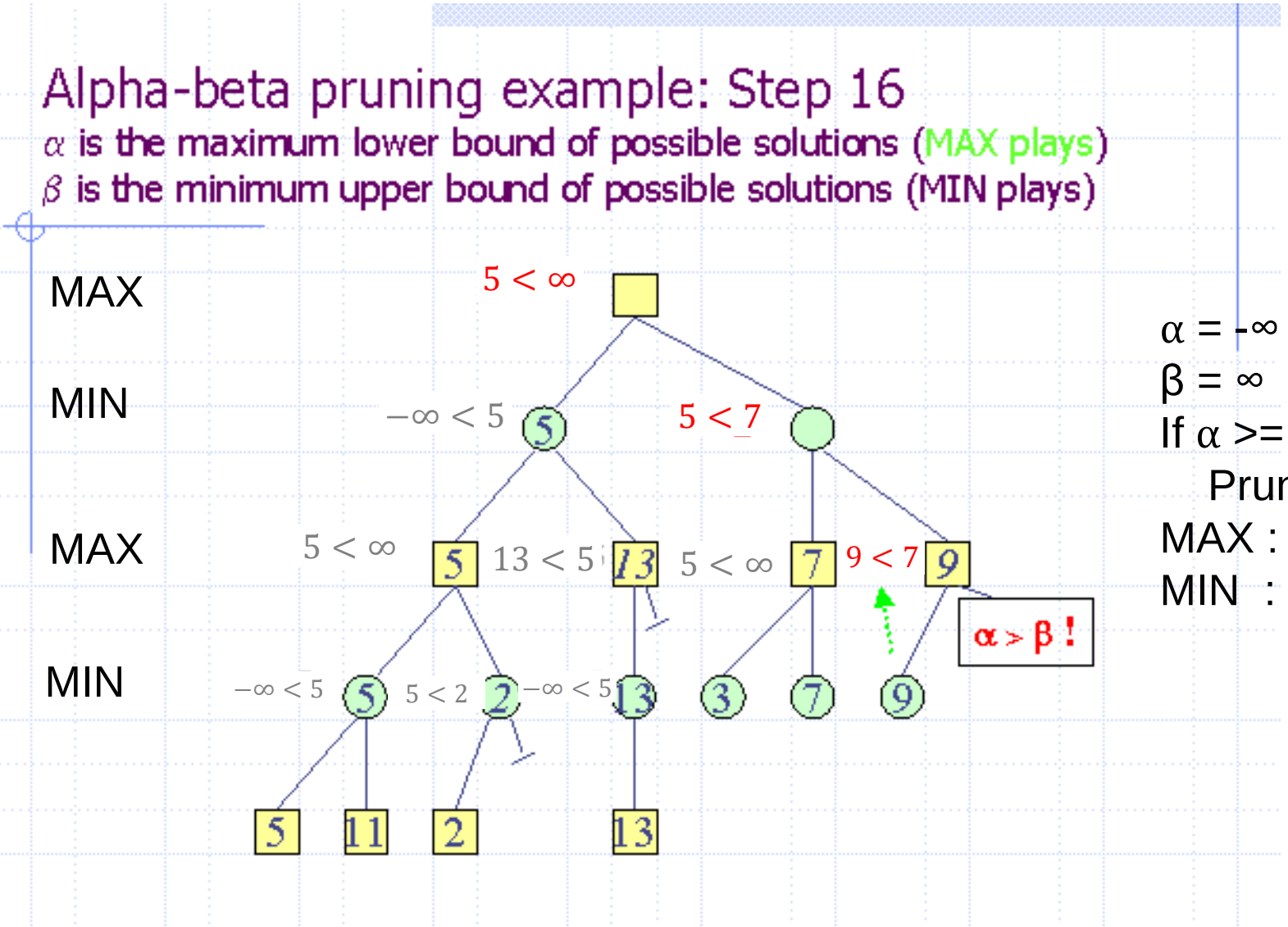


Alpha-Beta Pruning (cont.)



α = -∞
β = ∞
If α >= β then
Prune
MAX : Update α
MIN : Update β

Alpha-Beta Pruning (cont.)



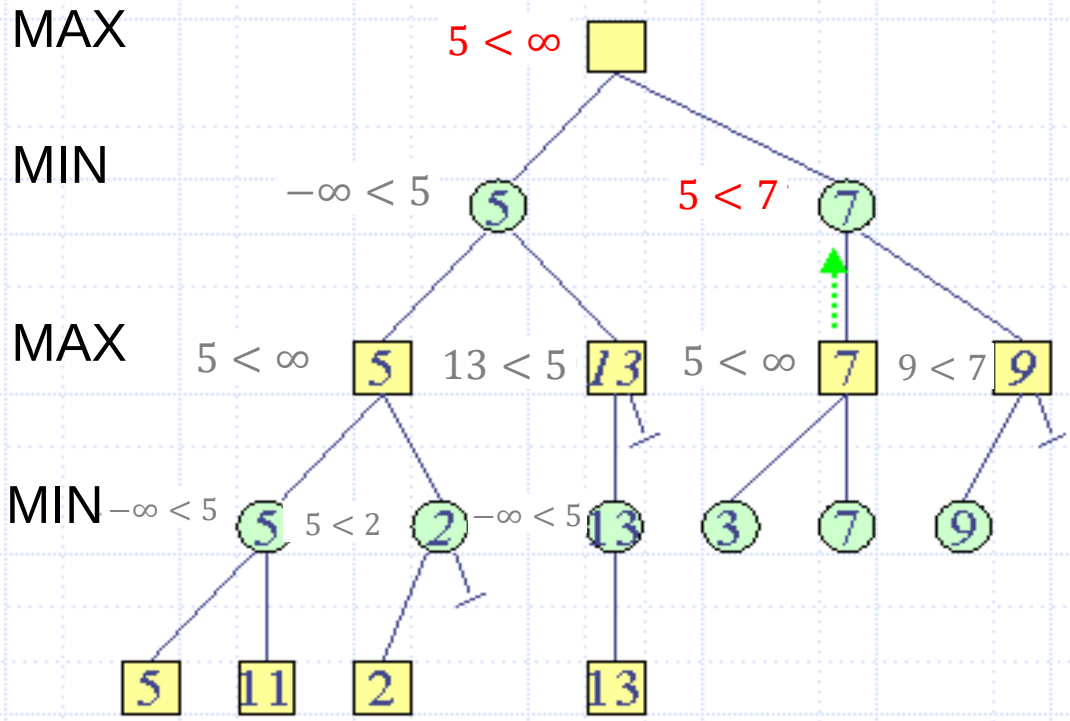
α = -∞
β = ∞
If α >= β then
Prune
MAX : Update α
MIN : Update β

Alpha-Beta Pruning (cont.)

Alpha-beta pruning example: Step 17

α is the maximum lower bound of possible solutions (MAX plays)

β is the minimum upper bound of possible solutions (MIN plays)



α = -∞

β = ∞

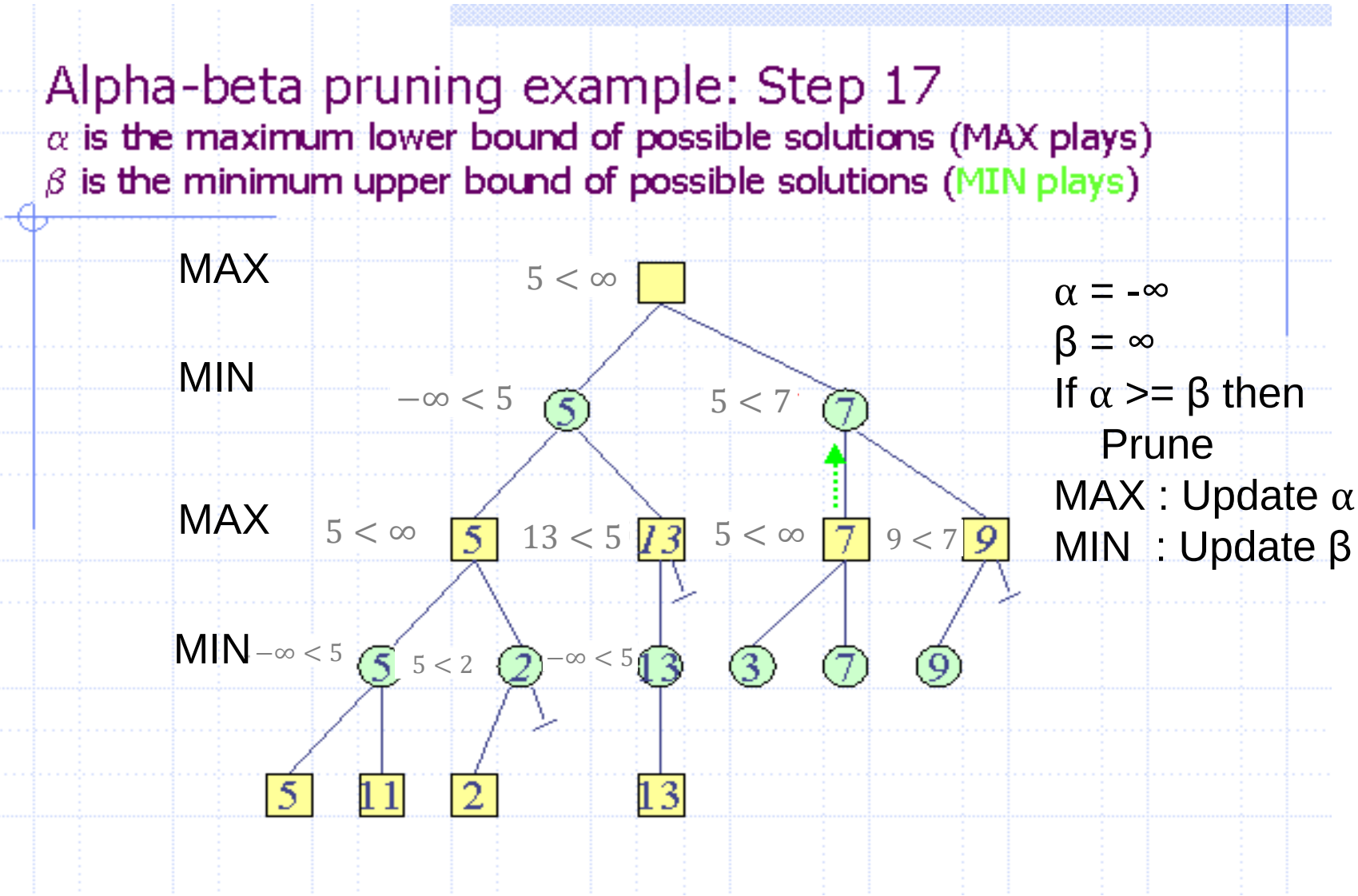
If α >= β then

Prune

MAX : Update α

MIN : Update β

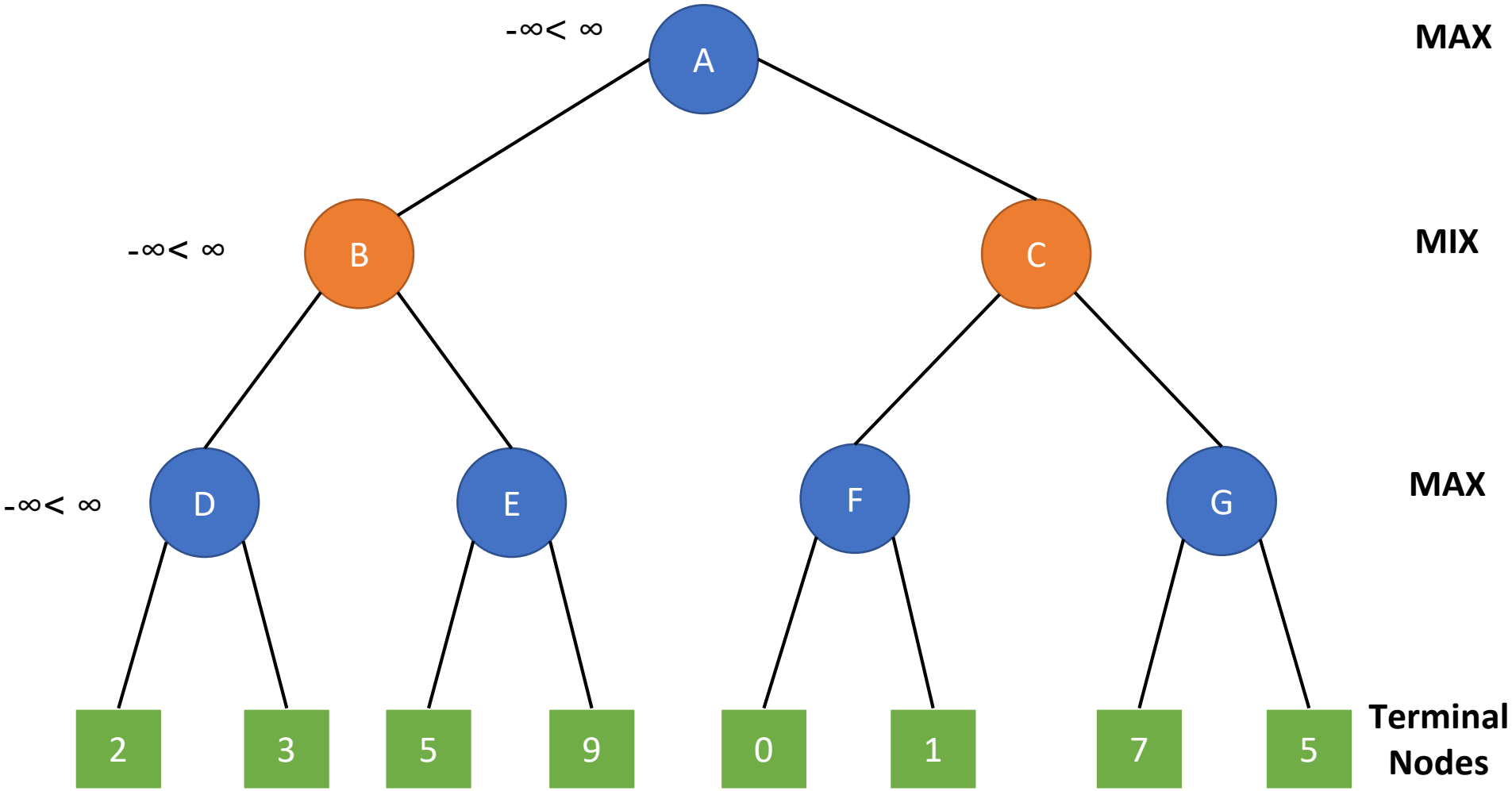
Alpha-Beta Pruning (cont.)



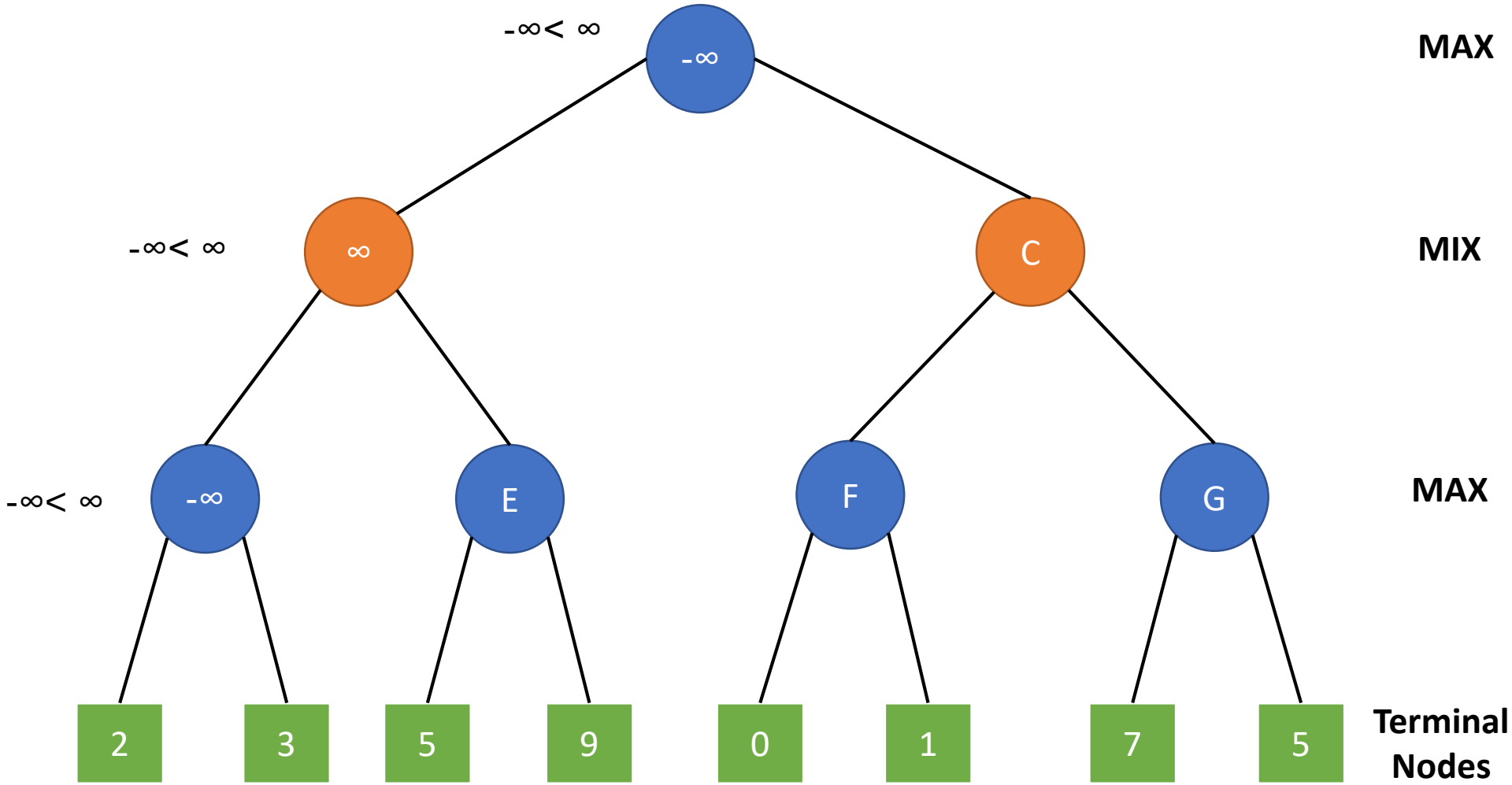
Alpha-Beta Pruning (cont.)

- The MAX only updates the value of alpha.
- The MIN only updates the value of beta.
- Start with $\alpha = -\infty$ and $\beta = \infty$
- If $\alpha >= \beta$ then prune
- The node values are passed to upper nodes, not alpha and beta values.
- Alpha and beta values are passed to child nodes.

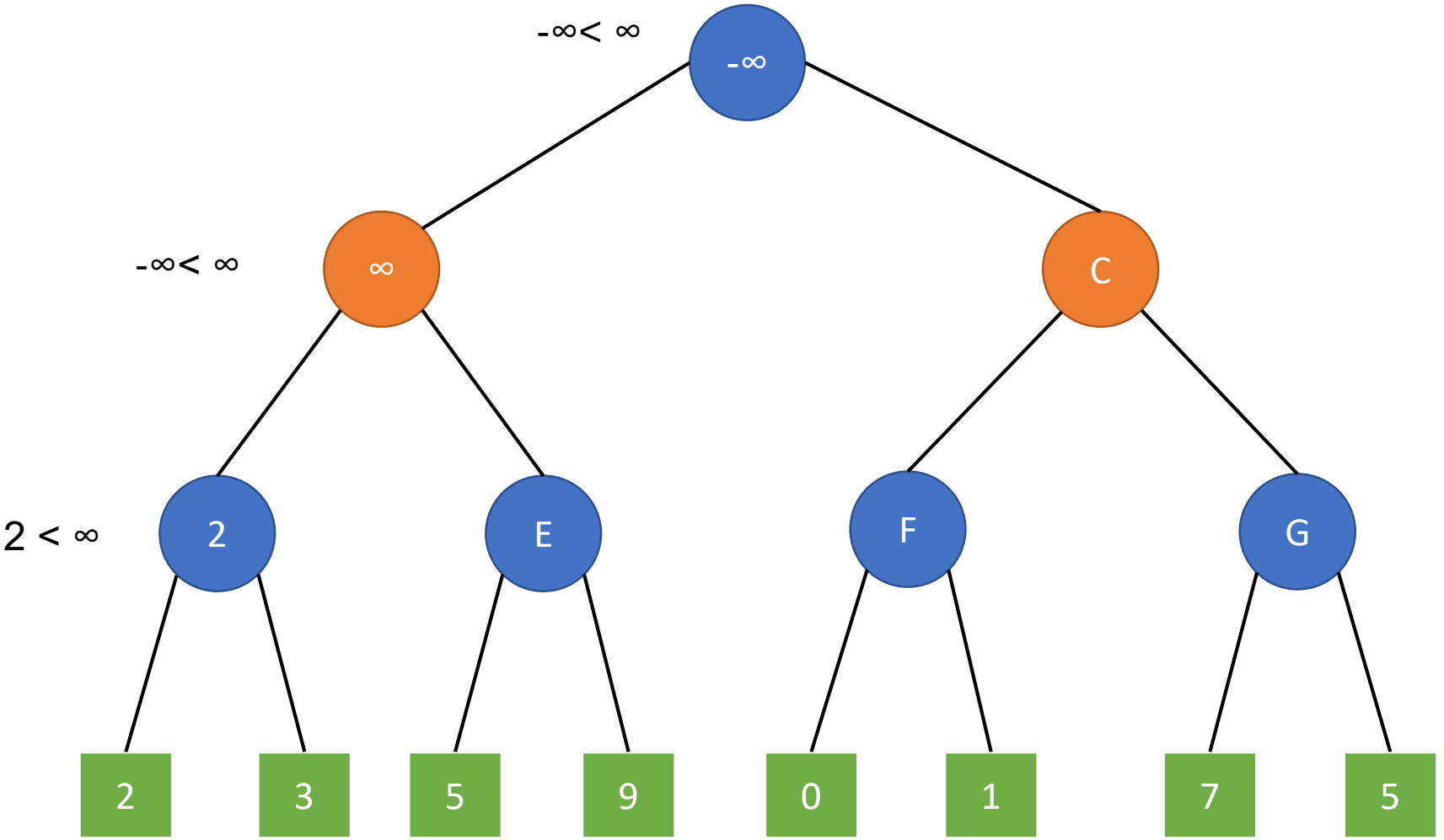
Alpha-Beta Pruning



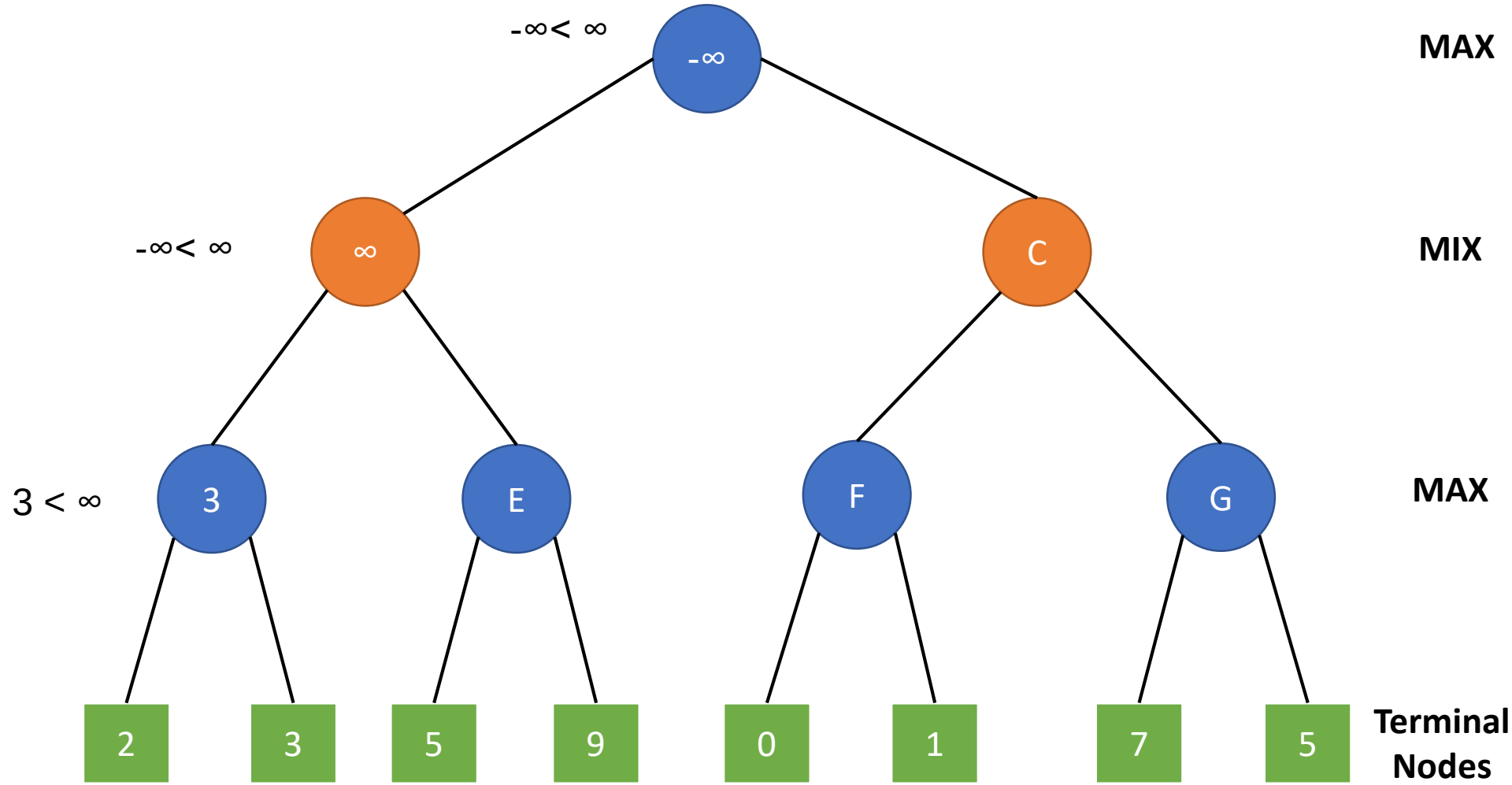
Alpha-Beta Pruning



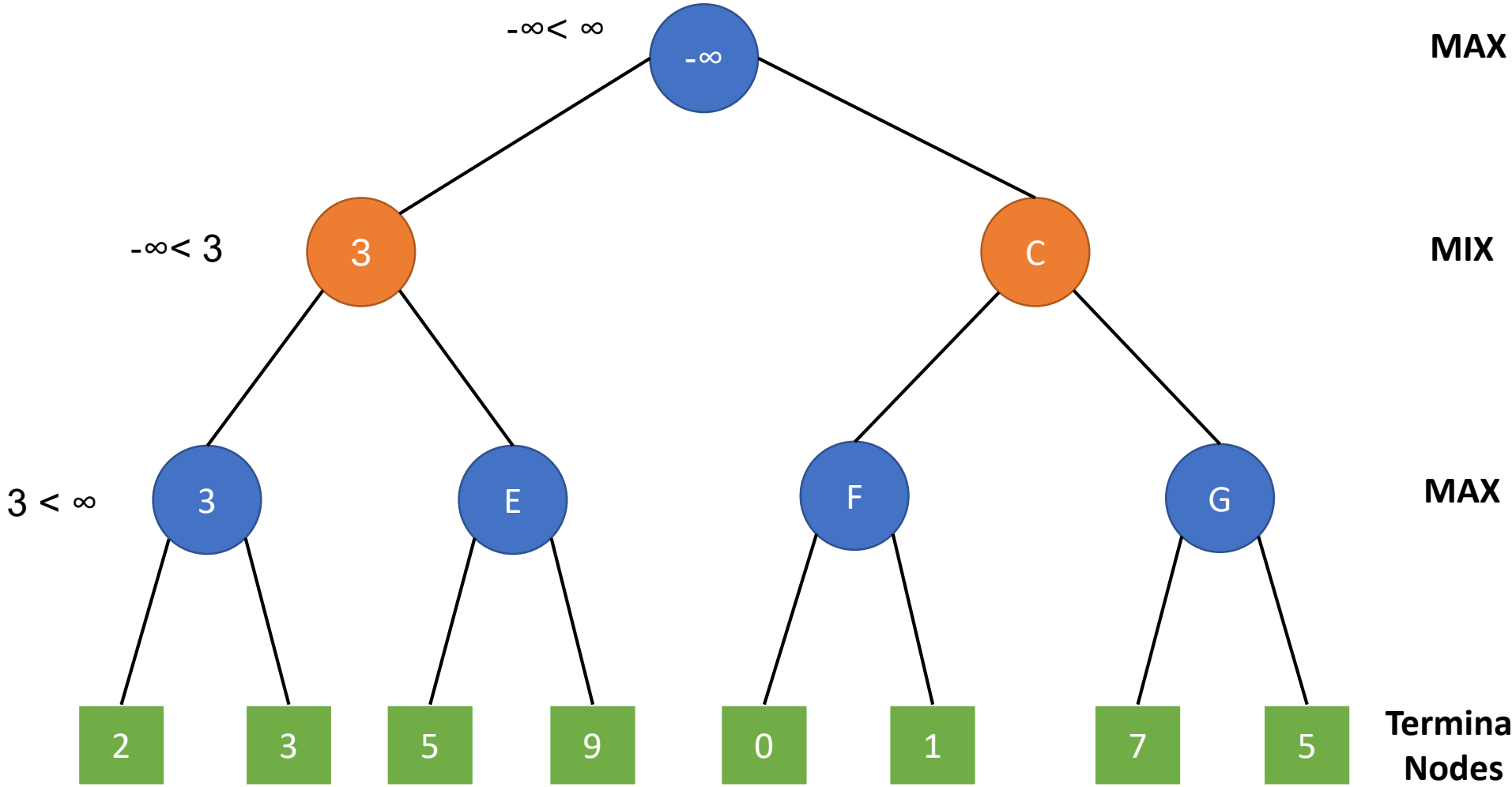
Alpha-Beta Pruning



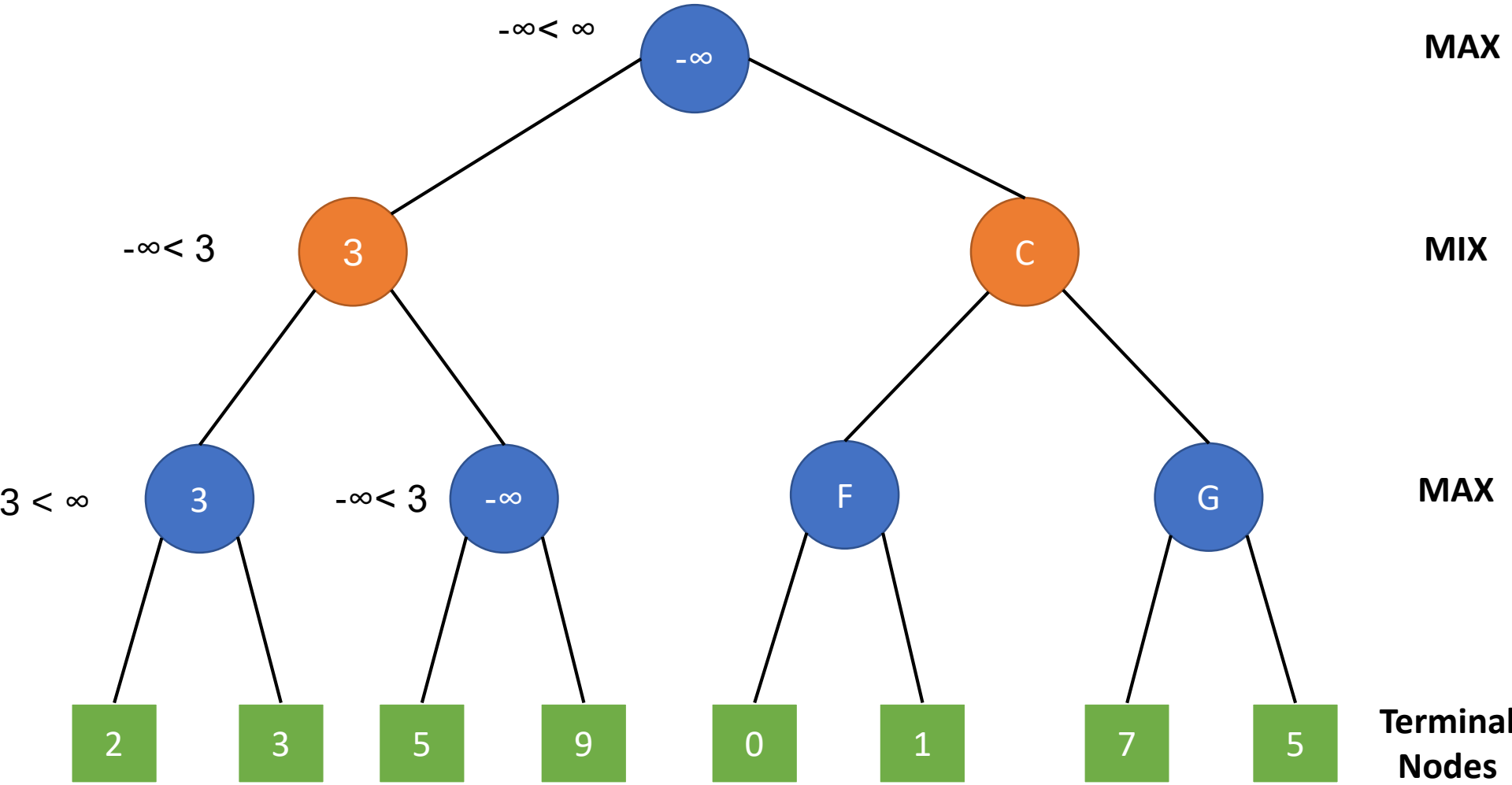
Alpha-Beta Pruning



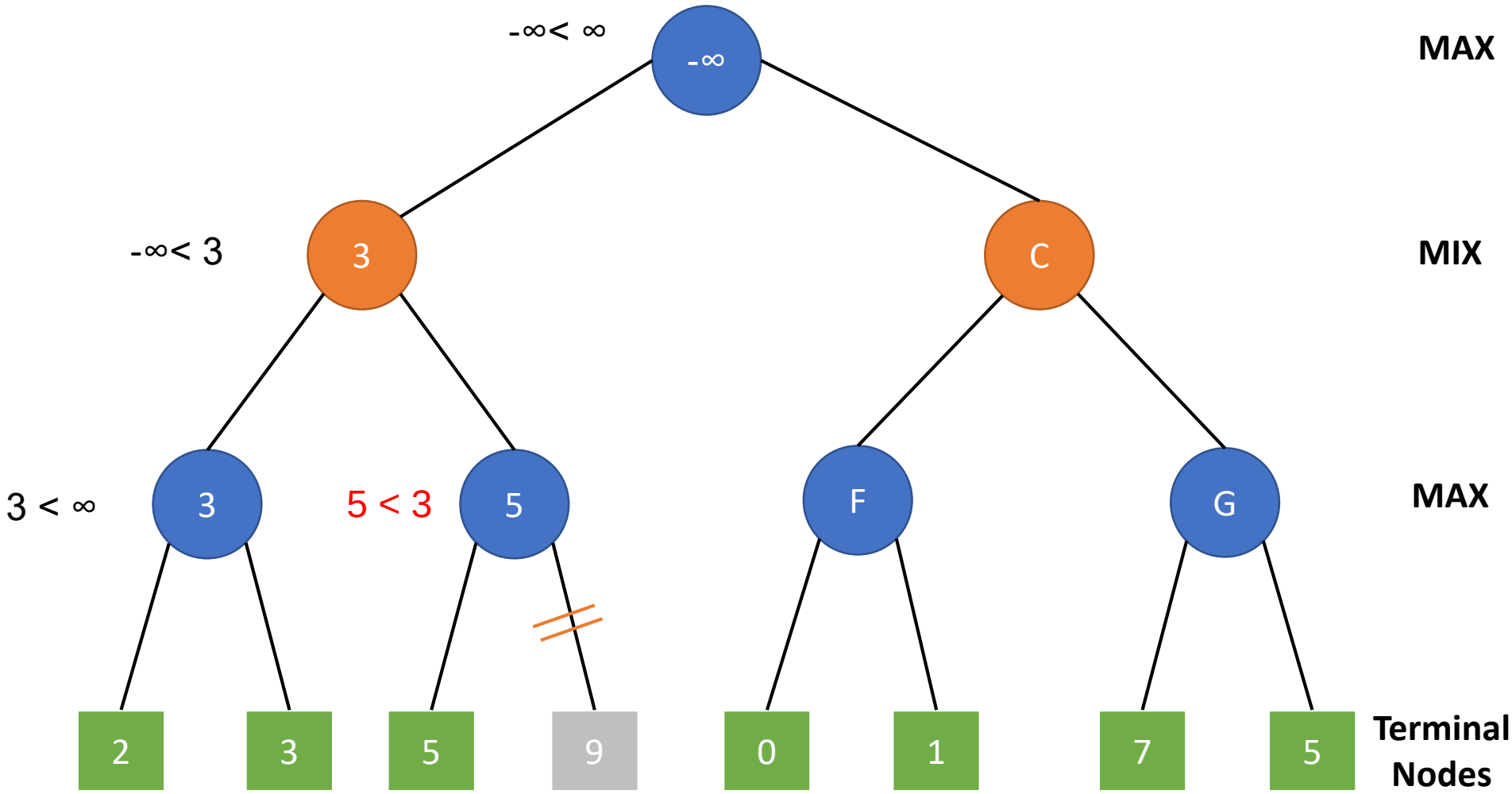
Alpha-Beta Pruning



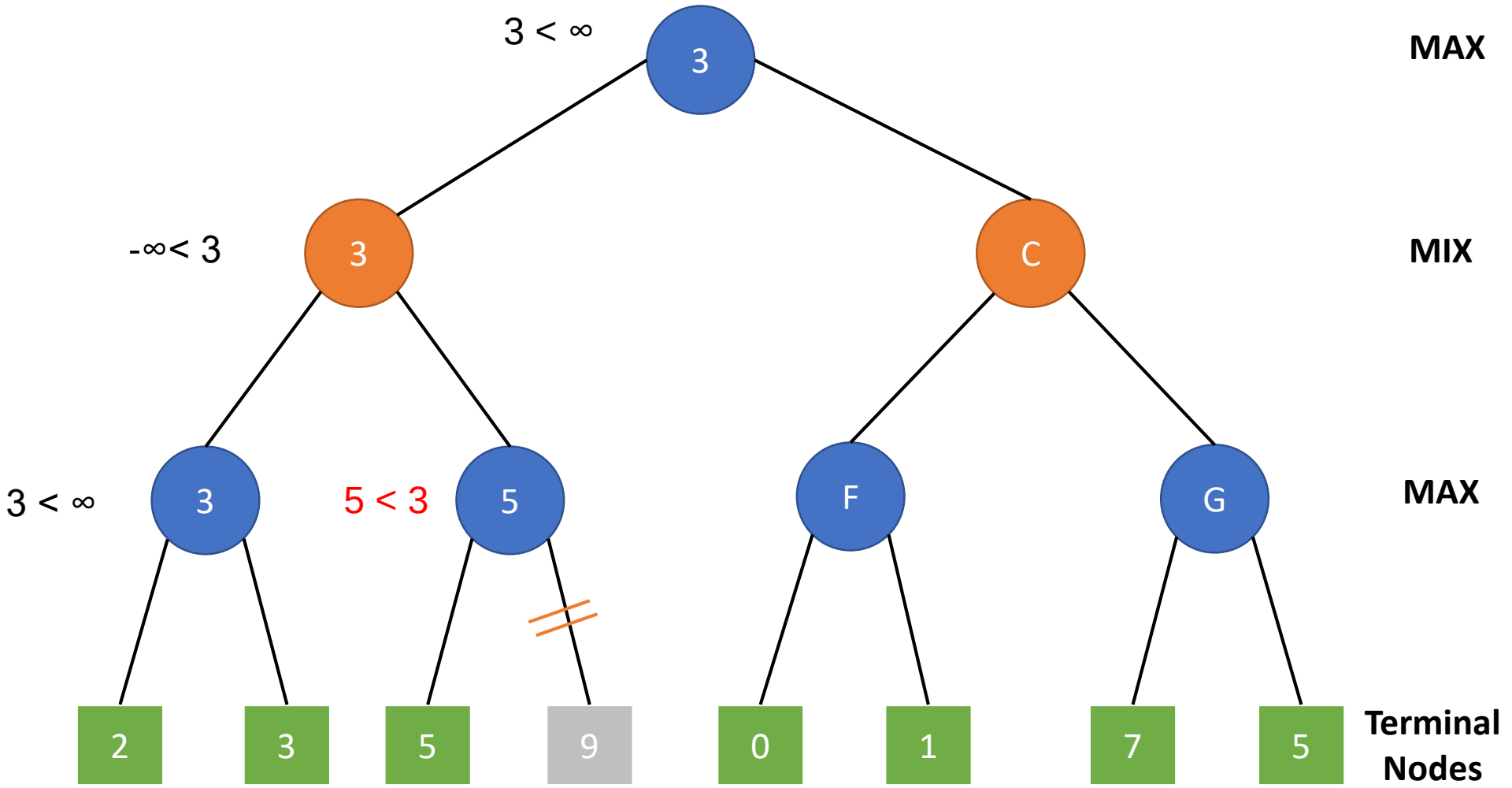
Alpha-Beta Pruning



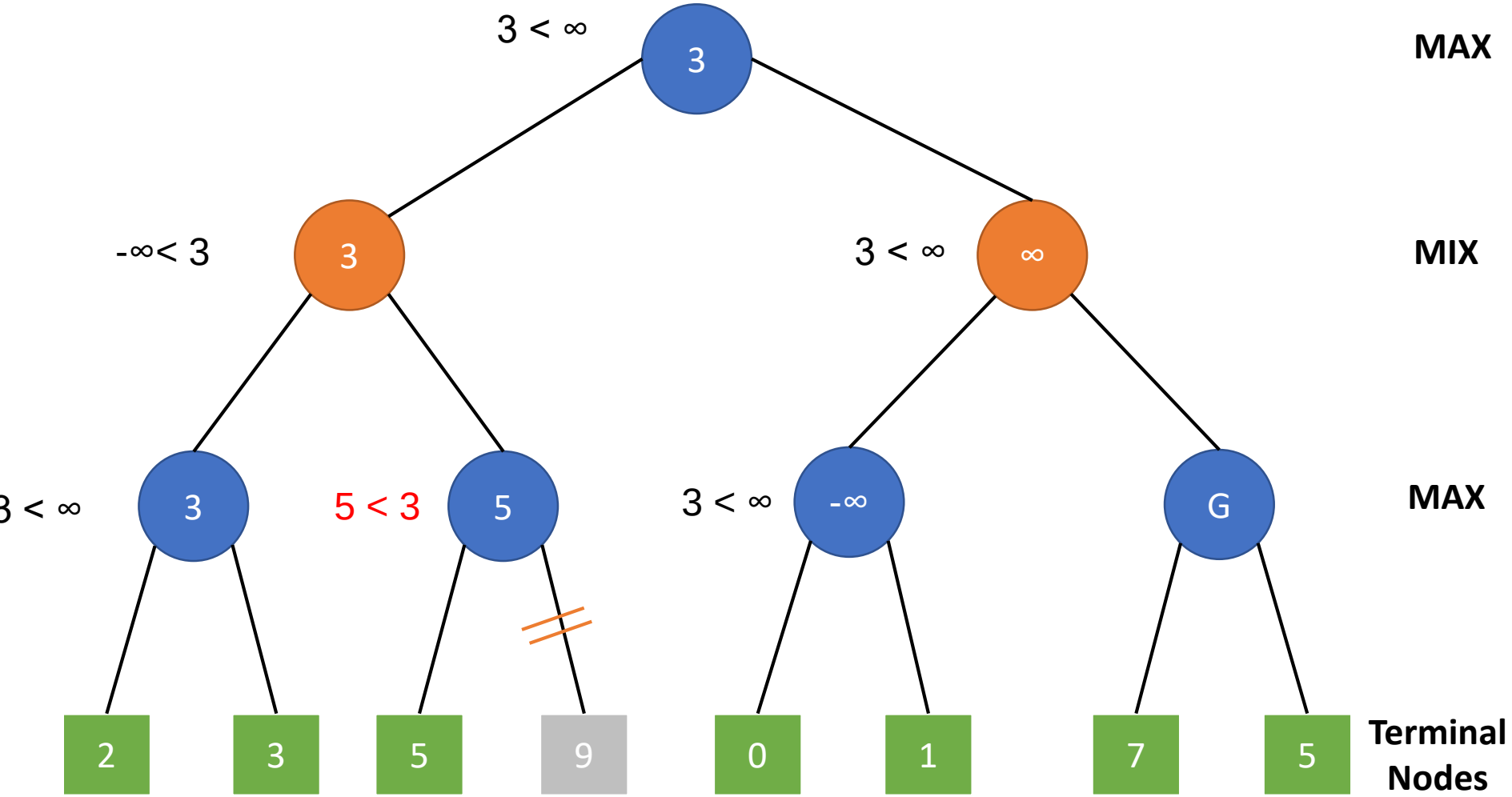
Alpha-Beta Pruning



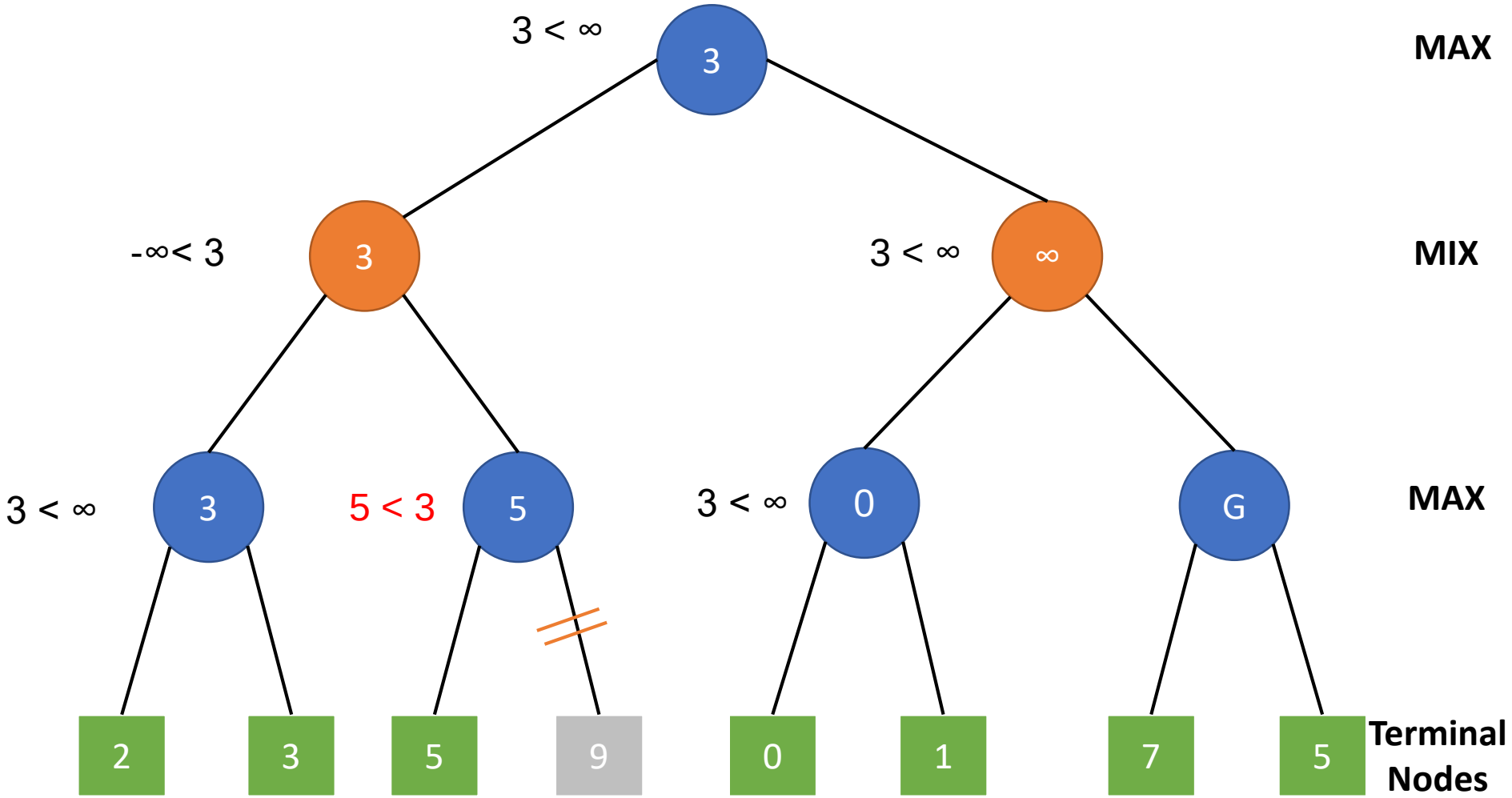
Alpha-Beta Pruning



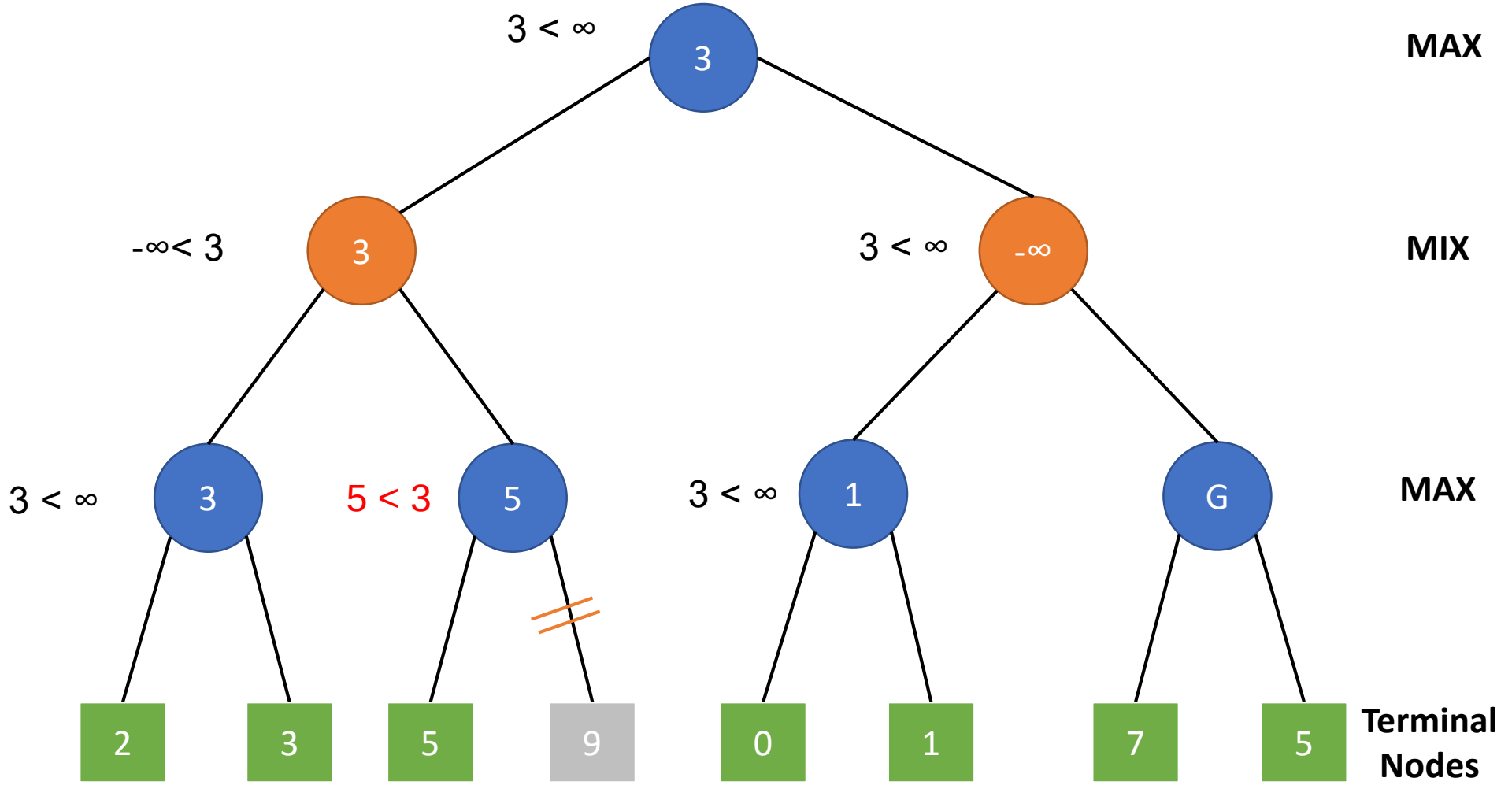
Alpha-Beta Pruning



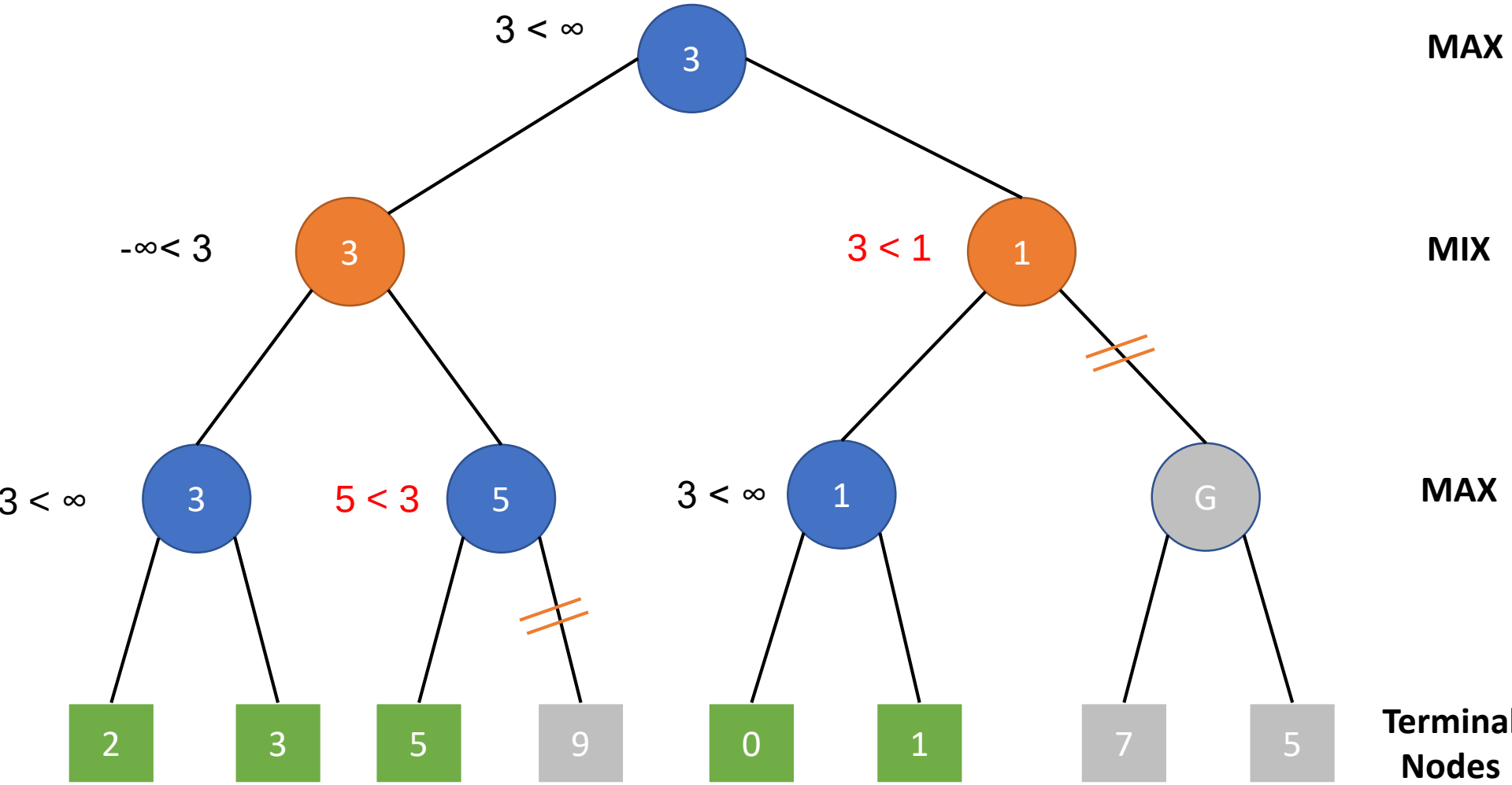
Alpha-Beta Pruning



Alpha-Beta Pruning



Alpha-Beta Pruning



Effectiveness of Alpha-Beta Pruning

- Guaranteed to compute same root value as Minimax
- Worst case: no pruning, same as Minimax ($O(b^d)$)
- Good child ordering improves effectiveness of pruning
- With "perfect ordering"- Best case :
  - Time complexity drops to $O(b^{d/2})$ nodes
  - Doubles solvable depth!
  - Full search of, e.g. chess, is still hopeless…
- Exact solution is impossible even for "small" games like chess
- For Deep Blue, alpha-beta pruning reduced the average branching factor from 35-40 to 6.

Resource Limits

- The **minimax algorithm** generates the entire game search space.
- The **alpha–beta algorithm** allows us to prune large parts of it.
  - alpha–beta still has to search all the way to terminal states for at least a portion of the search space. This depth is usually not practical.

- **Problem:** In realistic games, cannot search to leaves!
- **Solutions:**
  - Cutting off search: Considers all possible moves to a certain depth in the search tree
  - Forward pruning: Ignores moves that look bad, and follows promising lines “as far as possible.”

This slide from: ai.berkeley.edu

Heuristic Alpha–Beta Tree Search

- **Solution 1:** Depth-limited search
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function for non-terminal positions
  - Guarantee of optimal play is gone

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MIN}. \end{cases}$$

  - For terminal states: EVAL(s, p)=UTILITY(s, p)
  - For nonterminal states: the evaluation must be between a loss and a win: UTILITY(loss, p) ≤ EVAL(s, p) ≤ UTILITY(win, p)

This slide from: ai.berkeley.edu

Heuristic Alpha–Beta Tree Search

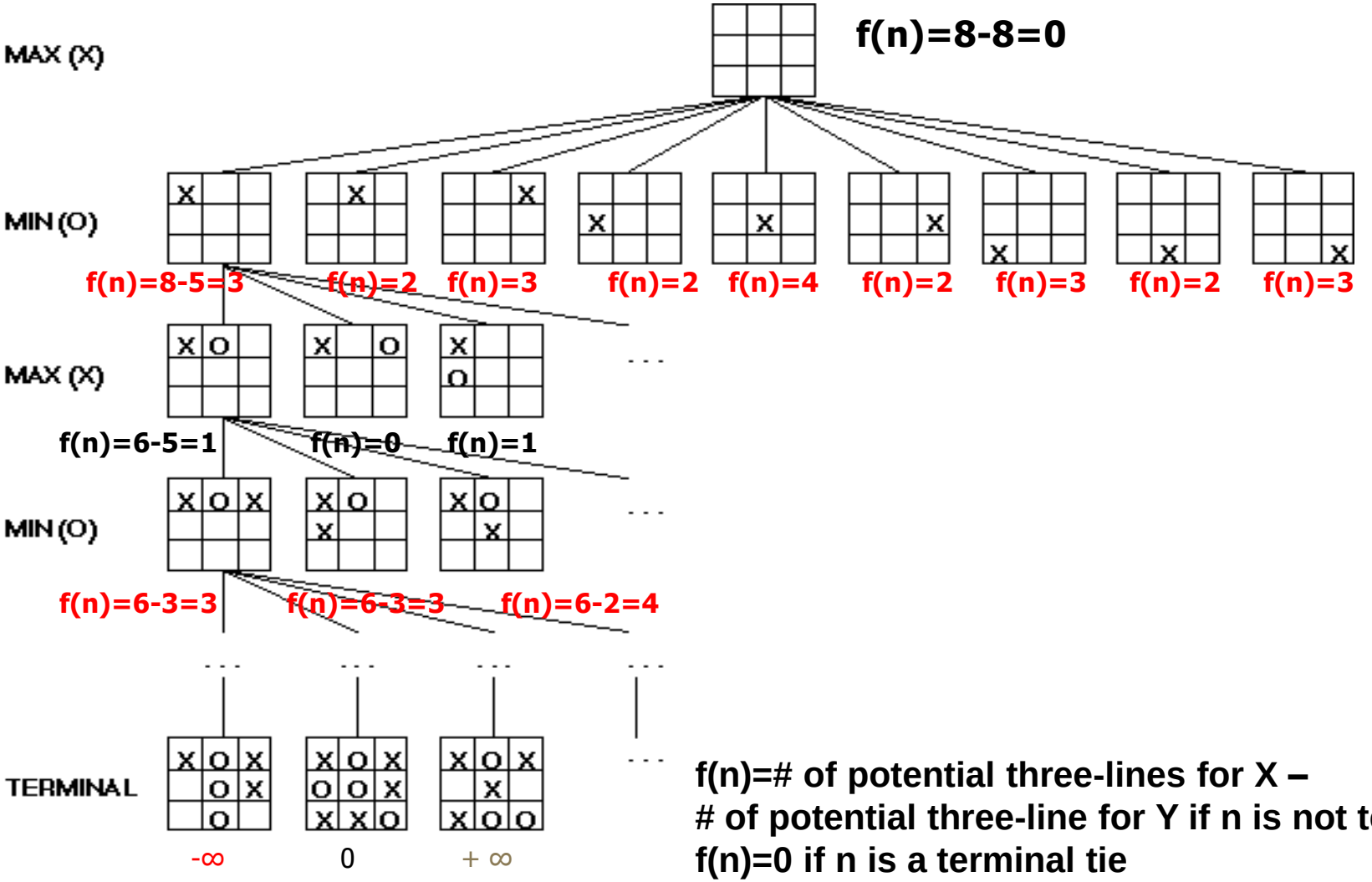
- Evaluation function: how good a ‘board position’ is
  - Based on *static features* of that board alone
  - effectively turning nonterminal nodes into terminal leaves.
  - An evaluation function returns an estimate of the expected utility of the game from a given position, just as the heuristic functions return an estimate of the distance to the goal.

Evaluation Functions

- **Zero-sum** assumption lets us use **one function** to describe goodness for both players.
  - f(n) > 0 if we are winning in position *n*
  - f(n) = 0 if position *n* is tied
  - f(n) < 0 if our opponent is winning in position *n*

- Build using **expert knowledge**,
  - Tic-tac-toe: f(n)=
    (# of 3 lengths open for me) - (# open for you)

A Partial Game Tree: Tic-Tac-Toe



f(n)=# of potential three-lines for X –
of potential three-line for Y if n is not terminal
f(n)=0 if n is a terminal tie
f(n)=+ ∞ if n is a terminal win
f(n)=- ∞ if n is a terminal loss

Chess Evaluation Functions

- Alan Turing's
  f(n)=(sum of your piece values)-(sum of opponent's piece values)

Pawn	**1.0**
Knight	**3.0**
Bishop	**3.25**
Rook	**5.0**
Queen	**9.0**

Table: Pieces values for a simple Turing-style evaluation function often taught to novice chess players.

- More complex: weighted sum of positional features:

$$\sum w_i \, feature_i(n)$$

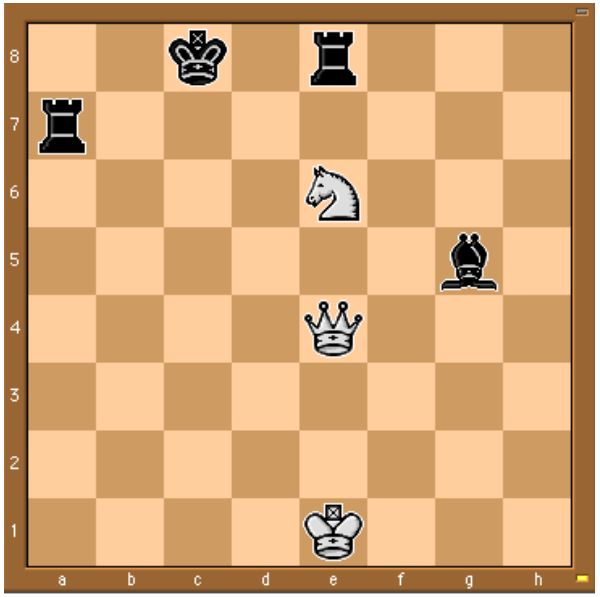
- Deep Blue has > 8000 features.

Positive: rooks on open files, knights in closed positions, control of the center, developed pieces.

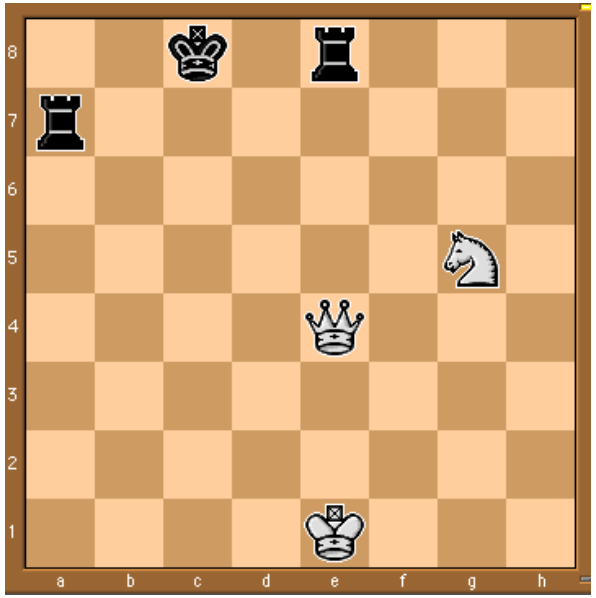
Negative: doubled pawns, wrong-colored bishops in closed positions, isolated pawns, pinned pieces.

Examples of more complex features.

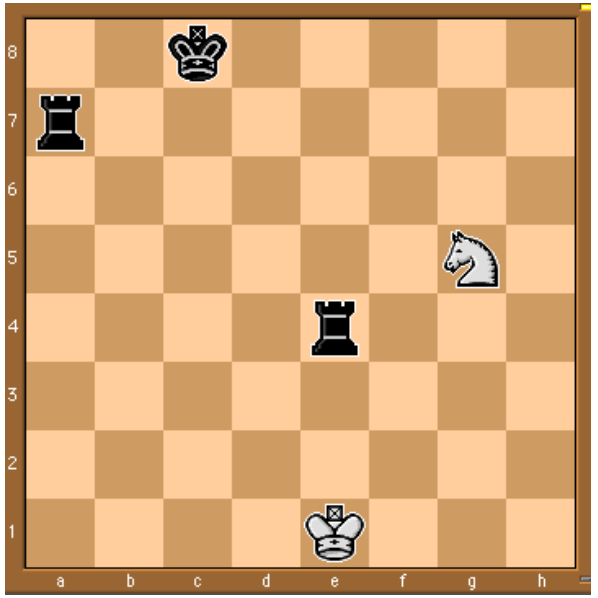
Some Chess Positions and Evaluations



White to move
f(n)=(9+3)-(5+5+3.25)
=-1.25



… Nxg5??
f(n)=(9+3)-(5+5)
=2



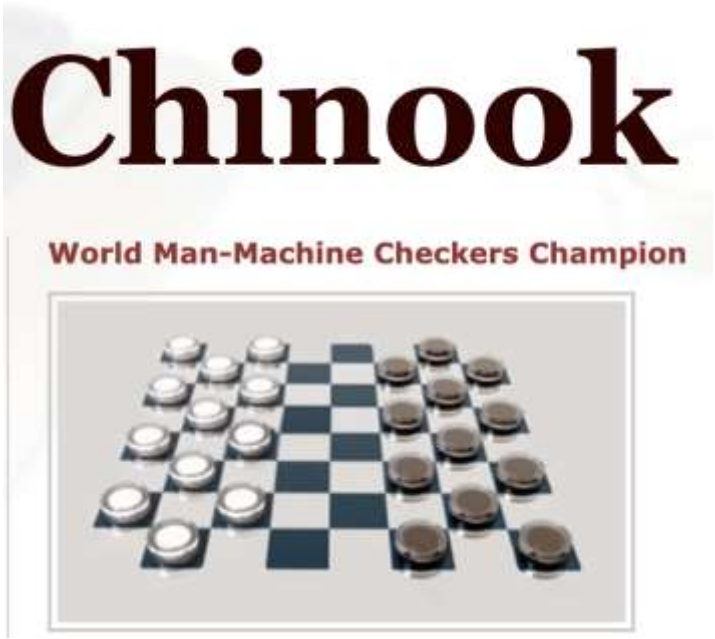
Uh-oh: Rxg4+
f(n)=(3)-(5+5)
=-7

And black may force checkmate

So, considering our opponent's possible responses would be wise.

Chinook and Deep Blue

- Chinook
  - the World Man-Made Checkers Champion, developed at the University of Alberta.
  - Competed in human tournaments, earning the right to play for the human world championship, and defeated the best players in the world.
  - Play Chinook at http://www.cs.ualberta.ca/~chinook



Chinook and Deep Blue

- Deep Blue
  - Deep Blue was the first computer to win a game
  - Defeated world champion Garry Kasparov 3.5-2.5 in 1997 after losing 4-2 in 1996.
  - Uses a parallel array of 256 special chess-specific processors
  - Evaluates 200 billion moves every 3 minutes; 12-ply search depth
  - Expert knowledge from an international grandmaster.
  - 8000 factor evaluation function tuned from hundreds of thousands of grandmaster games
  - Tends to play for tiny positional advantages.



(Image credit: Roger Celestin/Newscom)

Other Solutions

- Forward pruning:
  - prunes moves that appear to be poor moves, but might possibly be good ones.
  - Example: Beam Search
- Use experience (precompute tables):
  - Lookup table for best moves seen from past experiences.
  - Example: move pawn to e4 in Chess, or play x in center in tic-tac-toe
- Problem: what if the branching factor is huge (e.g. Go game)? Or its hard to compute a heuristic?

Monte Carlo Tree Search

Idea: use results of simulations to guide growth of the game tree

- MCTS strategy does not use a heuristic evaluation function.
- MCTS estimates the average utility over a number of simulations of complete games starting from the current state.

- Basic algorithm:
  - Play lots of random games starting with each possible move
  - Keep winning statistics for each move
  - Play move with best winning percentage

Monte Carlo Tree Search

Four Phases for the algorithm:

- **Selection** : Starting at the root node, MCTS moves down the tree using a selection rule.
- **Expansion** : When the selection phase reaches a leaf node that isn't terminal, the algorithm expands the tree by adding one or more child nodes representing possible actions from that state.
- **Simulation Phase**: From the newly added node, a random playout is performed until reaching a terminal state (**rollout**). During this phase, moves are chosen randomly or using simple heuristics, making the simulation computationally inexpensive.
- **Backpropagation Phase**: The result of the simulation is propagated back up the tree to the root, updating statistics (visit counts and win rates) for all nodes visited during the selection phase.

Monte Carlo Tree Search - UCB

Basic Selection rule : UCB = Upper confidence bound fourmla

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

- ***U(n)*** is is the total utility of all playouts that went through node ***n***
- ***N(n)*** is number of visits to node ***n***
- ***PARENT(n)*** number of simulations from the parent node
- This rule manages the **Exploitation** (average success of n so far) and **Exploration** (explore all nodes at least a little bit)
- ***C*** is the exploration constant - how important is exploration relative to exploitation?

Monte Carlo Tree Search

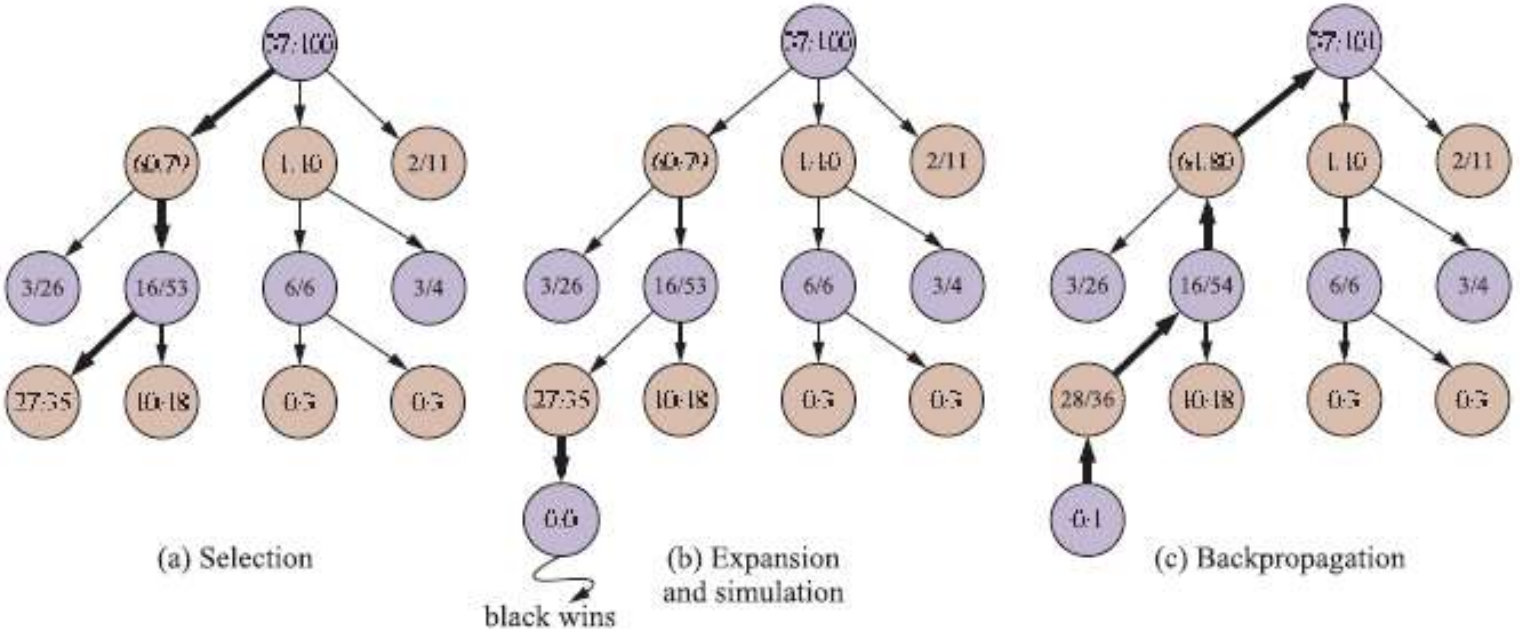
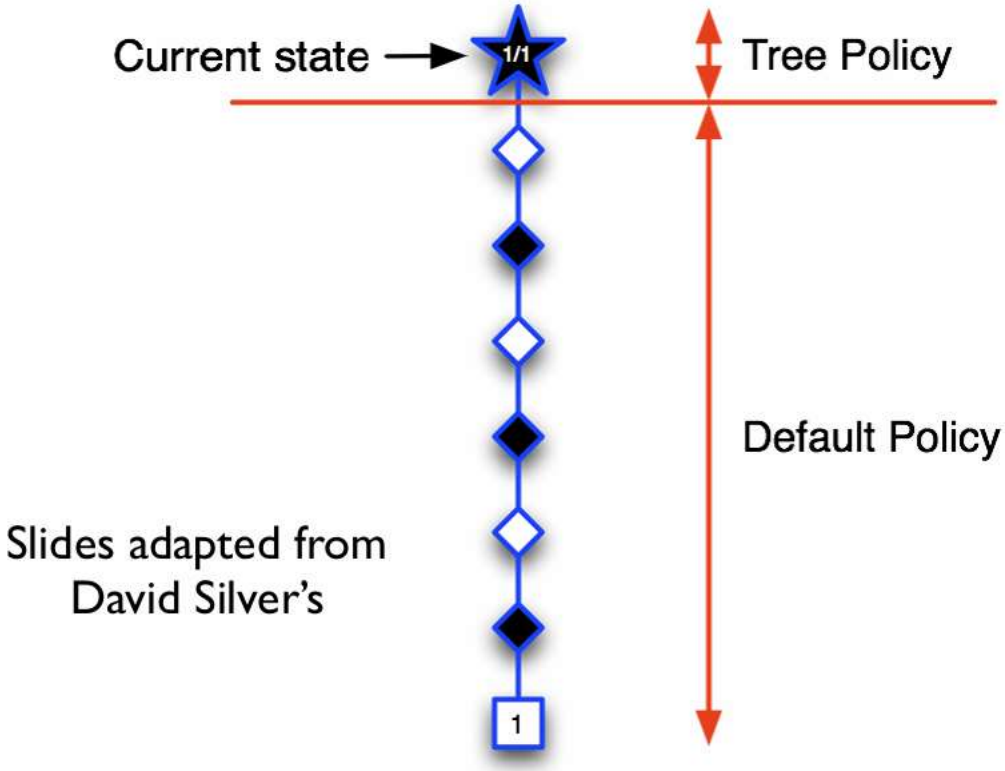


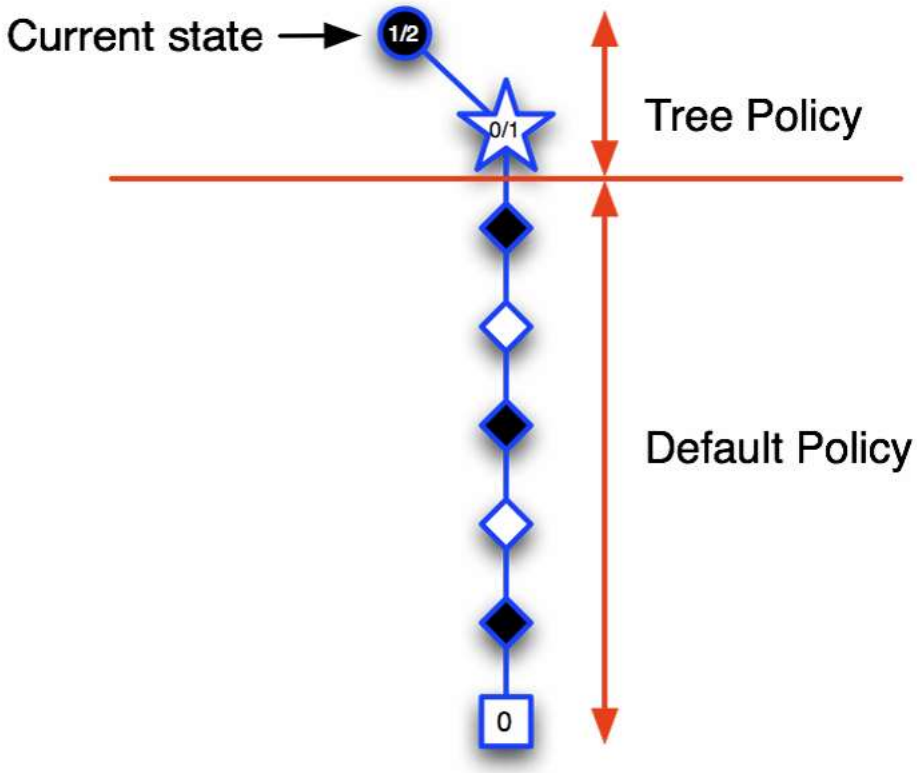
Figure 5.10 One iteration of the process of choosing a move with Monte Carlo tree search (MCTS) using the upper confidence bounds applied to trees (UCT) selection metric, shown after 100 iterations have already been done. In (a) we select moves, all the way down the tree, ending at the leaf node marked 27/35 (for 27 wins for black out of 35 playouts). In (b) we expand the selected node and do a simulation (playout), which ends in a win for black. In (c), the results of the simulation are back-propagated up the tree.

Monte Carlo Tree Search



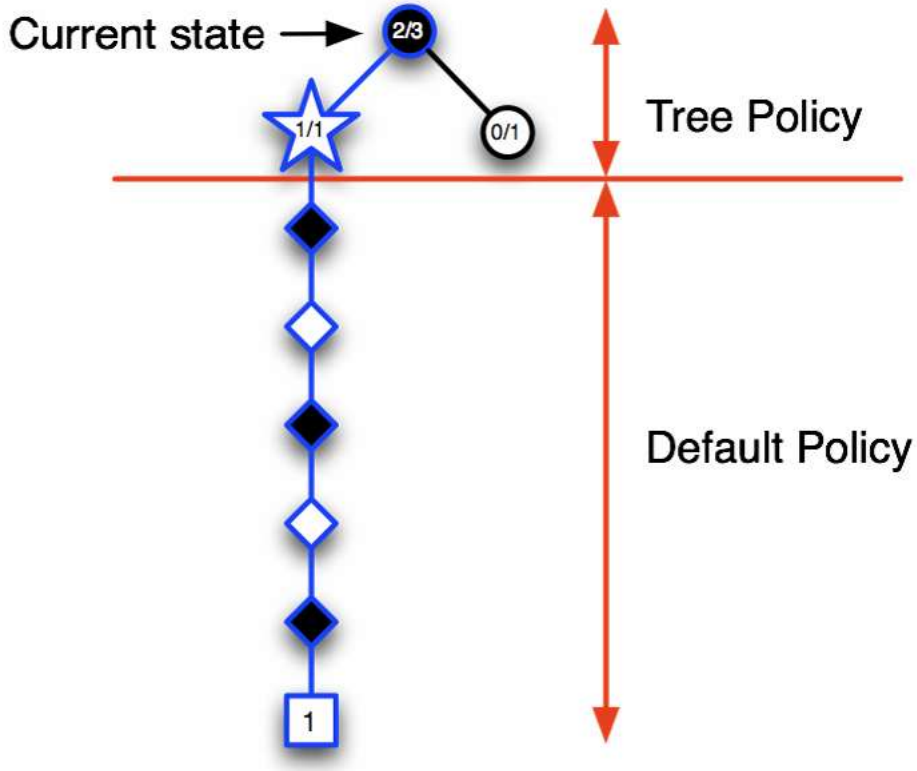
This slide from:web.stanford.edu

Monte Carlo Tree Search



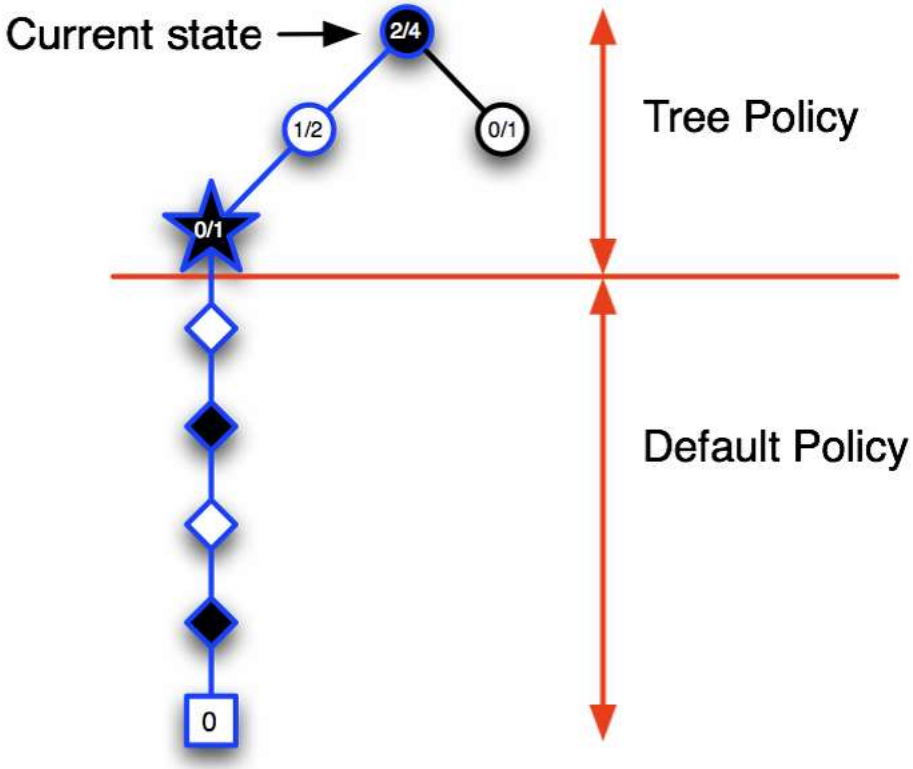
This slide from:web.stanford.edu

Monte Carlo Tree Search



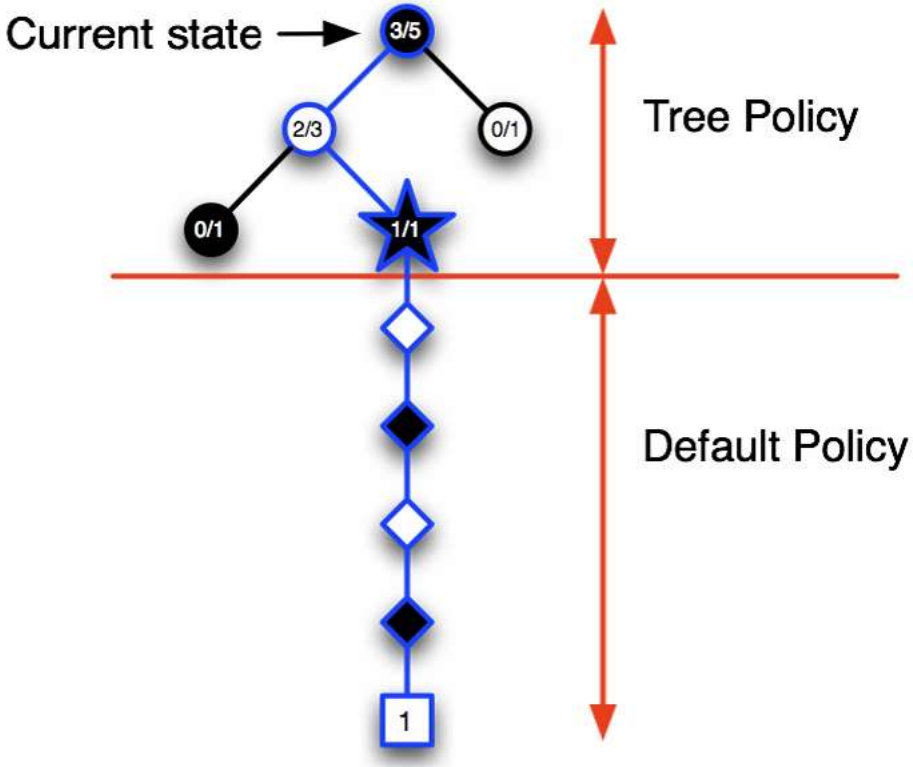
This slide from:web.stanford.edu

Monte Carlo Tree Search



This slide from:web.stanford.edu

Monte Carlo Tree Search



This slide from:web.stanford.edu

Monte Carlo Tree Search

- Very successful in practice
- Scales OK to parallel machines
- Why and how does it work?
  - Still poorly understood
- Limitation: stochastic nature of Monte Carlo search means it might fail to consider good moves.

This slide from:web.stanford.edu